

policygraph: the complete guide

A line-by-line teaching walk-through of the whole project, written for someone who has never seen the code.

Part 0. How to read this guide

This document explains everything in the repository: what the project is for, how the data flows, what every file does, and what every line of code in it means. It is long because the request was "every single thing". It is organised so that you can also dip in:

- **Part 1** explains the problem and the design in plain language, and defines every technical term used later.
- **Part 2** explains the tooling: how the project is built, run, tested and deployed.
- **Part 3** walks through the data pipeline (`policygraph/`) file by file, line by line.
- **Part 4** walks through the analysis modules (`policygraph/insights/`) and the web app (`policygraph/app.py`, `policygraph/views/`).
- **Part 5** walks through the extraction service (`extraction_service/`), the part that talks to a language model.
- **Part 6** explains the configuration files, the tests, the findings, and how to run, change and extend the project.

Code is shown in boxes with line numbers. Each box is followed by an explanation keyed to those line numbers. Where several lines do the same kind of thing (for example five `import` lines) they are explained together.

Conventions in the explanations:

- "The pipeline" means the Python package `policygraph/`, run stage by stage from the command line.
- "The service" means `extraction_service/`, a separate program with its own web API and a background worker.
- "The app" means the Streamlit web page in `policygraph/app.py` and `policygraph/views/`.
- "CDI" is the California Department of Insurance. "FAIR Plan" is California's state-mandated insurer of last resort. "SB 824" is the 2018 state law that creates one-year moratoriums on non-renewals after a wildfire emergency. "FHSZ" is CAL FIRE's Fire Hazard Severity Zone map. "ACS" is the Census Bureau's American Community Survey. "ZCTA" is the Census version of a ZIP code area.

Part 1. What the project is and how it thinks

1.1 The question

Home insurers in California have been refusing to renew policies ("non-renewals") at rising rates. The project asks one narrow question about one county: **what is actually happening with home-insurance non-renewals in Contra Costa County, and what appears to be driving it?**

The candidate explanations are the ones anyone would list:

1. **Fire hazard.** Insurers drop houses that are likely to burn.
2. **Price.** Expensive houses are expensive to rebuild, so insurers drop them regardless of fire risk.
3. **Moratoriums.** After a declared wildfire emergency, SB 824 forbids non-renewals in ZIP codes near the fire for one year. Do those moratoriums prevent non-renewals or only delay them?
4. **Regulation.** The Insurance Commissioner's rules and rate decisions change insurers' incentives. Does the timing of non-renewals follow the regulatory calendar more than the fire calendar?
5. **Actors.** Which people and organisations sit on the most paths between a decision and an outcome in the county?

None of these can be answered from any single data set. The answer lives in the *joins*: CDI's ZIP-level counts joined to CAL FIRE's hazard map; the text of CDI's moratorium bulletins joined to the count time series; FAIR Plan counts joined to both. The project is a machine for making those joins reliably, with a receipt for every fact.

1.2 The design principles

The whole codebase follows six rules, and understanding them makes every file predictable.

1. **Structured data never touches a language model.** Spreadsheets, PDF tables, map files and Census JSON are read by ordinary deterministic code. A model reads prose only (bulletins, regulations, press releases) and emits typed JSON.
2. **Canonical identifiers before fuzzy matching.** A ZIP code is `zip:94563`. An insurer is its NAIC company code, `insurer:naic:25143`. A bill is `bill:ca:SB824`. A fire is `fire:eaton:2025`. A moratorium is `moratorium:2025-01-07:eaton`. Only when a mention cannot be keyed like this does the project fall back to an alias table, then to embeddings, then to a model acting as a judge.
3. **Every edge carries provenance.** Each fact in the graph records where it came from (document, URL), the exact quoted text that supports it (the "span"), what produced it (deterministic code or a named model), and a confidence.

4. **The graph is temporal.** Facts have `valid_from` and `valid_to` dates. Nothing is overwritten; new facts are appended.
5. **The pipeline is idempotent and content-addressed.** Every stage is keyed by a hash of its inputs. Running a stage twice on the same inputs does nothing new. A changed input reprocesses only what depends on it.
6. **Insights are precomputed; serving is read-only.** The web app never calls a model or runs a heavy query. It reads small files that the pipeline wrote.

1.3 The data flow in one picture



Each arrow is one command (`make fetch`, `make normalize`, and so on). `make all` runs them in order.

1.4 The graph model

The graph has three tables (defined in `config/schema.sql`, explained in Part 6):

- **entities:** one row per thing: a ZIP, an insurer, a fire, a moratorium, a regulation, an official, a rate filing, a hazard class, the anonymous "voluntary market". Each has an id, a type, a readable name, and optional attributes (for ZIPs: county, median home value, median income).
- **edges:** one row per fact: `src --rel--> dst`, with dates, a JSON bag of properties, the document and span it came from, the extractor, and a confidence. The relation vocabulary is closed:

relation	from → to	produced by
NONRENEWED_IN	voluntary market → ZIP	CDI spreadsheets (deterministic), one edge per ZIP-year-release with counts in <code>props</code>
HAZARD_SHARE	ZIP → FHSZ class	geometry overlay (deterministic), <code>props.pct</code> is the area share
HAS_FAIR_PLAN_POLICIES	ZIP → FAIR Plan	FAIR Plan PDF (deterministic), <code>props.count</code> per fiscal year
PROTECTED_BY	ZIP → moratorium	bulletins (model + deterministic ZIP-block expansion)
TRIGGERED	fire → moratorium	bulletins
ISSUED	official → moratorium or regulation	bulletins and press releases
FILED	insurer → rate filing	press releases
DECIDED	official → rate filing	press releases
AUTHORED	official → regulation	bulletins ("Senate Bill 824 (Lara, ...)")
DONATED_TO	insurer → official	in the vocabulary; no source ingested yet

- **documents:** one row per prose document: id (hash of its text), source, URL, publication date, full text.

1.5 Glossary of technical terms

These terms recur throughout the code. Skim now, refer back later.

- **Python package / module.** A module is one `.py` file. A package is a folder of modules with an `__init__.py` file. `policygraph/insights/trend.py` is the module `policygraph.insights.trend`.
- **`import x / from x import y`.** Loads another module so its functions and constants can be used.
- **Type hints** such as `def f(x: int) -> str`. Annotations saying what a function takes and returns. Python does not enforce them; they document intent and let tools check it.
- **`dataclass`.** A decorator that turns a class into a simple record with named fields. `frozen=True` makes instances immutable.
- **`Pydantic BaseModel`.** Like a `dataclass` but *validating*: constructing one from a dictionary checks every field's type and constraints and raises if anything is off. Used at every boundary where data comes from outside (the model's JSON, HTTP requests).
- **`pandas DataFrame / Series`.** A table and a column, respectively. Most transformations here are one-line pandas expressions.
- **`Parquet`.** A compressed columnar file format for tables; fast to read, keeps column types.
- **`DuckDB`.** An embedded SQL database that lives in one file (`data/graph.duckdb`) and queries pandas frames and parquet directly.
- **`GeoPandas / shapely / CRS`.** GeoPandas is pandas with a geometry column. A CRS (coordinate reference system) says what the numbers mean; EPSG:3310 is "California

Albers", a metre-based projection in which areas are correct; EPSG:4326 is plain latitude/longitude for web maps.

- **Regular expression (regex).** A pattern for matching text, compiled with `re.compile`. `\d{5}` means five digits; `^` and `$` anchor to line start and end; `(...)` captures a group; `(?:...)` groups without capturing; `?` makes the preceding thing optional; `re.I` ignores case; `re.M` makes `^/$` match at every line.
 - **SHA-256.** A hash function: any bytes in, a fixed 64-character hex string out, and any change to the input changes the output. Used to name files by content and to build stable ids.
 - **HTTP conditional GET.** Asking a web server "send me this file unless it is unchanged since this ETag / date". If unchanged the server answers `304 Not Modified` with no body.
 - **LLM (large language model).** The model that reads prose. The project uses an open-weight model (`gpt-oss-120b`) through Fireworks' OpenAI-compatible API, at temperature 0 (least random).
 - **JSON schema / structured output.** A description of the exact JSON shape the model must produce; the API refuses anything else.
 - **Embedding.** A vector of numbers representing a piece of text so that similar texts have similar vectors. Cosine similarity between two vectors is 1.0 for identical direction, 0 for unrelated.
 - **OLS regression.** Ordinary least squares: fit a line (or hyperplane) through points; the residual is how far each point sits above or below the fitted line. R^2 is the share of variance the fit explains.
 - **Betweenness centrality.** For a node in a network, the share of shortest paths between all other node pairs that pass through it.
 - **Streamlit.** A Python library that turns a script into a web page; `st.dataframe`, `st.selectbox` and so on draw widgets.
 - **uv.** The Python package manager used here; `uv run` runs a command inside the project's virtual environment.
 - **Make.** A tool that runs named recipes from a `Makefile`; `make fetch` runs the `fetch` recipe.
-

Part 2. Tooling: how the project is built and run

2.1 Makefile

```
1 .PHONY: all fetch normalize extract resolve graph insights app test eval lint service worker docker
2
3 PY := uv run --env-file .env python
4 PORT ?= 8000
5
6 all: fetch normalize extract resolve graph insights
7
8 fetch:      ; $(PY) -m policygraph fetch $(if $(FORCE),--force)
9 normalize:  ; $(PY) -m policygraph normalize
10 extract:   ; $(PY) -m policygraph extract $(if $(FORCE),--force)
11 resolve:   ; $(PY) -m policygraph resolve $(if $(FORCE),--force)
12 graph:     ; $(PY) -m policygraph graph
13 insights:  ; $(PY) -m policygraph insights
14 app:       ; uv run streamlit run policygraph/app.py
15 test:      ; uv run pytest -q
16 eval:      ; $(PY) -m extraction_service.eval
17 lint:      ; uv run ruff check . && uv run ruff format --check .
18 docker:    ; docker build -t policygraph .
19 service:   ; uv run --env-file .env uvicorn extraction_service.api:app --port $(PORT)
20 worker:    ; $(PY) -m extraction_service.worker
```

- **1** `.PHONY` tells Make these names are commands, not files to build, so `make test` always runs even if a file called `test` exists.
- **3** `PY` is a variable: "run Python inside the project environment, first loading variables from `.env`". `.env` holds secrets (API keys); it is never committed.
- **4** `PORT ?= 8000` sets a default port for the service; `make service PORT=8001` overrides it.
- **6** `make all` runs the six pipeline stages in order.
- **8–13** Each stage runs `python -m policygraph <stage>`, which goes through `policygraph/__main__.py` (Part 3). `$(if $(FORCE),--force)` appends `--force` only when you type `make fetch FORCE=1`.
- **14** `make app` starts the web page locally.
- **15–17** Tests, the extraction quality evaluation, and the linter/formatter check.
- **18** Builds the Docker image.
- **19** Starts the extraction service's HTTP API with uvicorn (a web server for Python). The `--env-file .env` matters: the API's resolve endpoint needs the model keys.
- **20** Starts the extraction worker, the only process that ever talks to a model.

2.2 pyproject.toml

```

1 [project]
2 name = "policygraph"
3 version = "0.1.0"
4 requires-python = ">=3.12"
5 dependencies = [
6     "duckdb>=1.1", "pandas>=2.2", "pyarrow>=17", "geopandas>=1.0", "openpyxl>=3.1",
7     "httpx>=0.27", "pydantic>=2.8", "pyyaml>=6", "typer>=0.12", "pypdf>=5",
8     "networkx>=3.3", "statsmodels>=0.14", "streamlit>=1.38", "pydeck>=0.9",
9     "fastapi>=0.115", "uvicorn>=0.30", "anthropic>=0.40",
10    "cryptography>=43", # FAIR Plan PDFs are AES-encrypted; pypdf needs it to decrypt
11 ]
12
13 [project.optional-dependencies]
14 dev = ["pytest>=8", "ruff>=0.6"]
15
16 [tool.ruff]
17 line-length = 130
18 target-version = "py312"
19
20 [tool.ruff.lint]
21 select = ["E", "F", "I", "B", "UP", "SIM"]
22
23 [tool.pytest.ini_options]
24 testpaths = ["tests"]
25 pythonpath = ["."]

```

- **1–4** Project name, version, and the Python version it needs.
- **5–11** Runtime dependencies. Each earns its place: `duckdb` (graph store), `pandas/pyarrow` (tables, parquet), `geopandas` (map overlay), `openpyxl` (reading `.xlsx`), `httpx` (HTTP client), `pydantic` (validation), `pyyaml` (config files), `typer` (command-line interface), `pypdf` (PDF text), `networkx` (graph algorithms), `statsmodels` (regression), `streamlit/pydeck` (app and map), `fastapi/uvicorn` (service API), `anthropic` (one of two model adapters), `cryptography` (the FAIR Plan PDF is encrypted).
- **13–14** Extra tools only developers need: `pytest` runs tests, `ruff` lints and formats.
- **16–21** Ruff settings: lines may be up to 130 characters; the selected rule families are errors (E), pyflakes (F), import ordering (I), bugbear (B), modern syntax (UP) and simplifications (SIM).
- **23–25** Pytest looks in `tests/` and puts the repo root on the import path so tests can `import policygraph`.

`uv.lock` (not shown) pins the exact version of every dependency so every machine installs the same thing.

2.3 `requirements.txt` (for Streamlit Cloud only)

```

streamlit>=1.48
pandas>=2.2
pyarrow>=17
pydeck>=0.9
altair>=5
numpy>=2

```


Streamlit Community Cloud installs only this file. The app imports nothing from the pipeline beyond path constants, so it needs none of the heavy dependencies (no DuckDB, no GeoPandas, no model client).

2.4 Dockerfile and .dockerignore

```
1 FROM python:3.12-slim
2 COPY --from=ghcr.io/astral-sh/uv:0.8 /uv /bin/uv
3 WORKDIR /app
4 ENV UV_LINK_MODE=copy UV_COMPILE_BYTECODE=1
5 COPY pyproject.toml uv.lock ./
6 RUN uv sync --frozen --no-dev --no-install-project
7 COPY . .
8 RUN uv sync --frozen --no-dev
9 CMD ["uv", "run", "streamlit", "run", "policygraph/app.py", "--server.port=8501", "--server.address=0.0.0.0"]
```

- **1** Start from a small official Python 3.12 image.
- **2** Copy the `uv` binary in from its own image.
- **5–6** Copy only the dependency files first and install dependencies. Docker caches this layer, so editing code does not reinstall everything.
- **7–8** Copy the rest of the project and finish the install.
- **9** The container's default command runs the app on port 8501, listening on all interfaces.

`.dockerignore` keeps `.git`, the virtual environment, raw and normalized data, `.env` and caches out of the image.

2.5 .github/workflows/ci.yml

```
name: ci
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: astral-sh/setup-uv@v5
      - run: uv sync --extra dev
      - run: make lint
      - run: make test
```

On every push and pull request GitHub checks out the code, installs `uv` and the dependencies (including dev tools), then runs the linter and the tests. A red mark on a commit means one of those failed.

2.6 .env.example and .gitignore

`.env.example` documents every environment variable. Copy it to `.env` and fill in the keys:

- `CENSUS_API_KEY`: needed once, by `make fetch`, for the ACS download.

- `EXTRACTION_SERVICE_URL`: where the pipeline finds the service (`http://localhost:8001` on this machine).
- `EXTRACTION_STORE`: where the service writes artifacts (`data/extractions`).
- `EXTRACTION_PROVIDER`: `openai_compat` or `anthropic`; which model adapter the worker uses.
- `OPENAI_COMPAT_BASE_URL`, `OPENAI_COMPAT_API_KEY`, `EXTRACTION_MODEL`: the OpenAI-compatible endpoint (Fireworks), its key, and the model id.
- `ANTHROPIC_API_KEY`: only if the provider is `anthropic`.
- `EMBED_MODEL`: the embedding model for the resolver's kNN step.
- `EXTRACTION_MAX_TOKENS`: output cap per model call (100,000; a 700-ZIP bulletin with quoted spans is about 80,000 tokens).

`.gitignore` keeps out of git: `.env`, the virtual environment, compiled Python, the raw downloads (330 MB), the normalized tables, the DuckDB file, the service's SQLite queue index, and macOS `.DS_Store` files. Everything else, including the extraction artifacts and the insight tables is committed, which is what lets the graph and the app rebuild without any network or key.

2.7 The command-line entry point: `policygraph/__main__.py`

```

1  import typer
2
3  from policygraph import extract, fetch, graph, insights, normalize, resolve
4
5  app = typer.Typer(no_args_is_help=True)
6  for name, mod in (
7      ("fetch", fetch),
8      ("normalize", normalize),
9      ("extract", extract),
10     ("resolve", resolve),
11     ("graph", graph),
12     ("insights", insights),
13 ):
14     app.command(name)(mod.run)
15
16 if __name__ == "__main__":
17     app()
```

- **1** Typer builds command-line interfaces from Python functions.
- **3** Import the six stage modules.
- **5** Create the CLI; with no arguments it prints help.
- **6–14** For each (name, module) pair, register the module's `run` function as a sub-command with that name. Typer reads the function's parameters, so `run(force: bool = False)` automatically becomes a `--force` flag.
- **16–17** When the file is executed as a script (`python -m policygraph`), start the CLI.

2.8 Shared constants: `policygraph/__init__.py`

```

1 from pathlib import Path
2
3 ROOT = Path(__file__).resolve().parent.parent
4 CONFIG, DATA, DOCS = ROOT / "config", ROOT / "data", ROOT / "docs"
5 RAW, NORMALIZED, EXTRACTIONS, INSIGHTS = (DATA / d for d in ("raw", "normalized", "extractions", "insights"))
6 GRAPH_DB = DATA / "graph.duckdb"
7 COUNTY_FIPS = "06013"

```

- **3** `__file__` is this file's path; two `.parent`s up is the repository root. Every path in the project derives from this, so the code works from any working directory.
- **4–6** Named folders and the graph file.
- **7** Contra Costa's FIPS county code, kept for reference.

2.9 Small helpers: `policygraph/util.py`

```

9 def sha256(data: bytes) -> str:
10     return hashlib.sha256(data).hexdigest()
11
12
13 def stable_id(*parts: Any) -> str:
14     return sha256(json.dumps(parts, sort_keys=True, default=str).encode())
15
16
17 def load_yaml(path: Path) -> dict:
18     return yaml.safe_load(path.read_text())
19
20
21 def dump_yaml(path: Path, obj: object) -> None:
22     path.write_text(yaml.safe_dump(obj, sort_keys=False))

```

- **9–10** Hash bytes to a hex string.
- **13–14** Hash any list of values: serialise them to JSON with sorted keys (so the same content always gives the same string; `default=str` turns dates and other non-JSON values into strings), then hash. Used for edge ids.
- **17–22** Read and write YAML files. `safe_load` refuses to execute anything; `sort_keys=False` preserves the order the code wrote.

Part 3. The pipeline, file by file

3.1 policygraph/fetch.py: downloading sources

```
1 import json
2 import os
3 from dataclasses import asdict, dataclass
4
5 import httpx
6
7 from policygraph import CONFIG, RAW
8 from policygraph.util import dump_yaml, load_yaml, sha256
9
10 MANIFEST = RAW / "manifest.yaml"
11 HASHES = CONFIG / "source_hashes.yaml" # url -> sha256 of the last fetched bytes, committed for review
12 EXT = {"xlsx": "xlsx", "pdf": "pdf", "shapefile_zip": "zip", "json": "json", "arcgis_geojson": "geojson", "html": "html"}
13 PAGE = 2000
14 UA = {"User-Agent": "policygraph/0.1 (+httpx)"} # insurance.ca.gov rejects the default httpx agent
```

- **1–8** Standard library (`json`, `os`, `dataclasses`), the HTTP client, and the project's paths and helpers.
- **10** The local manifest: for every URL, where its bytes landed and what validators the server gave. Lives in `data/raw/`, which is not committed.
- **11** The committed list of URL → hash, so a reviewer can see when an upstream file changed.
- **12** Maps a source type (from `sources.yaml`) to the file extension to save under.
- **13** ArcGIS feature services return at most a few thousand features per request; this is the page size.
- **14** CDI's web server rejects requests without a browser-like User-Agent header.

```
17 @dataclass(frozen=True)
18 class Fetched:
19     source: str
20     url: str
21     path: str
22     sha: str
23     etag: str | None = None
24     last_modified: str | None = None # HTTP Last-Modified, or the ArcGIS layer's dataLastModifiedDate
```

One record per downloaded file: which source, which URL, where it is, its hash, and the two "validators" the server may have sent. An ETag is an opaque version token; Last-Modified is a date. Either lets us ask "has this changed?" without downloading.

```

27 def conditional(prior: Fetched | None) -> dict[str, str]:
28     if prior is None:
29         return {}
30     return {k: v for k, v in (("If-None-Match", prior.etag), ("If-Modified-Since", prior.last_modified)) if v}

```

Builds the request headers for a conditional GET from a previous fetch: `If-None-Match` carries the ETag, `If-Modified-Since` the date; only the ones we have are included. With no prior fetch there is nothing to send.

```

33 def download(url: str, prior: Fetched | None) -> tuple[bytes, str | None, str | None] | None:
34     """None when the server answers 304 to the prior validators, i.e. the bytes on disk are
    still current."""
35     with httpx.Client(timeout=300, follow_redirects=True, headers=UA | conditional(prior))
    as c:
36         r = c.get(url)
37         if r.status_code == 304:
38             return None
39         r.raise_for_status()
40         return r.content, r.headers.get("etag"), r.headers.get("last-modified")

```

- **35** Open an HTTP client with a five-minute timeout, following redirects, sending our User-Agent merged (`|`) with the conditional headers.
- **36–38** Make the request. Status 304 means "not modified": return `None` and keep what is on disk.
- **39** Any other error status (404, 500...) raises. The project's rule is to fail loudly.
- **40** Otherwise return the bytes plus the server's validators for next time.

```

43 def download_arcgis(layer: str, prior: Fetched | None) -> tuple[bytes, str | None, str | None] | None:
44     feats: list[dict] = []
45     with httpx.Client(timeout=300, headers=UA) as c:
46         edited = str(c.get(layer, params={"f": "json"}).raise_for_status().json()["editingInfo"]["dataLastEditDate"])
47         if prior is not None and prior.last_modified == edited:
48             return None
49         while True:
50             params = {"where": "1=1", "outFields": "*", "outSR": 3310, "resultOffset": len(feats), "resultRecordCount": PAGE}
51             r = c.get(f"{layer}/query", params=params | {"f": "geojson"})
52             r.raise_for_status()
53             page = r.json()["features"]
54             feats += page
55             if len(page) < PAGE:
56                 return json.dumps({"type": "FeatureCollection", "features": feats}).encode(), None, edited

```

CAL FIRE's hazard zones come from an ArcGIS "feature service", which is queried, not downloaded.

- **46** First ask the layer for its metadata; `editingInfo.dataLastEditDate` is when its data last changed. That timestamp plays the role of Last-Modified.
- **47–48** If it matches what we recorded last time, nothing changed.

- **49–56** Otherwise page through every feature: `where=1=1` means all rows, `outSR=3310` asks for California Albers coordinates, `result0ffset` moves through the pages. When a page comes back short, we have everything; wrap the features as one GeoJSON `FeatureCollection` and return the bytes with the edit date as the validator.

```

59 def expand(url: str) -> str:
60     out = os.path.expandvars(url)
61     if "${" in out:
62         raise ValueError(f"unset env var in {url}")
63     return out

```

The ACS URL contains `${CENSUS_API_KEY}`. `expandvars` substitutes environment variables; if any `${` survives, the variable was unset and we stop with a clear error rather than sending a broken request.

```

66 def fetch_one(source: str, url: str, kind: str, prior: Fetched | None, force: bool) -> tuple[Fetched, str]:
67     if prior is not None and not os.path.exists(prior.path):
68         prior = None
69     if prior is not None and not force and not (prior.etag or prior.last_modified):
70         return prior, "cached" # no validator to ask the server about; FORCE=1 re-download
71     got = (
72         download_arcgis(url, None if force else prior)
73         if kind == "arcgis_geojson"
74         else download(expand(url), None if force else prior)
75     )
76     if got is None:
77         return prior, "unchanged"
78     body, etag, last_modified = got
79     sha = sha256(body)
80     path = RAW / source / f"{sha}.{EXT[kind]}"
81     if not path.exists():
82         path.parent.mkdir(parents=True, exist_ok=True)
83         path.write_bytes(body)
84     status = "new" if prior is None else "ok" if prior.sha == sha else "changed"
85     return Fetched(source, url, str(path), sha, etag, last_modified), status

```

The decision tree for one URL:

- **67–68** If the manifest mentions a file that no longer exists on disk, forget the prior record.
- **69–70** If we have the file but the server gave no validator last time (the Census API and CDI's HTML pages do not), keep the file and report `cached`. `FORCE=1` overrides this.
- **71–75** Otherwise download conditionally (`None if force else prior` disables the conditional headers under force). ArcGIS layers take the paging path.
- **76–77** A `None` means 304: report `unchanged`.
- **78–83** New bytes: hash them and save them under `data/raw/<source>/<hash>.<ext>`. Because the name is the hash, the same bytes never get written twice and a changed file never overwrites an old one.
- **84–85** Report `new`, `ok` (same hash as before) or `changed`.

```

88 def run(force: bool = False) -> None:
89     cfg = load_yaml(CONFIG / "sources.yaml")["sources"]
90     prior = {e["url"]: Fetched(**e) for entries in (load_yaml(MANIFEST) if MANIFEST.exists()
91     else {}).values() for e in entries}
92     pinned = {h["url"]: h["sha"] for h in load_yaml(HASHES)} if HASHES.exists() else {}
93     manifest: dict[str, list[dict]] = {}
94     for name, spec in cfg.items():
95         urls = spec.get("urls") or ([spec["url"]] if spec.get("url") else [])
96         if not urls:
97             print(f"skip {name}: url not set")
98             continue
99         manifest[name] = []
100         for u in urls:
101             fetched, status = fetch_one(name, u, spec["type"], prior.get(u), force)
102             manifest[name].append(asdict(fetched))
103             if pinned.get(u, fetched.sha) != fetched.sha:
104                 status = "changed" # differs from the committed hash: review before the ne
105 xt normalize
106         print(f"{status:9} {name} {fetched.sha[:12]}")
107     RAW.mkdir(parents=True, exist_ok=True)
108     dump_yaml(MANIFEST, manifest)
109     dump_yaml(HASHES, [{"url": e["url"], "sha": e["sha"]} for entries in manifest.values()
110     for e in entries])

```

- **89** Read the source registry.
- **90** Load the previous manifest into a dictionary keyed by URL (empty on first run).
- **91** Load the committed hashes.
- **93–104** For each source and each of its URLs: fetch, record, and print one status line. If the hash differs from the committed one, the line says `changed` even when the local file was already current, so a reviewer notices upstream drift.
- **105–107** Write the local manifest and the committed hash list.

3.2 `policygraph/normalize/__init__.py`: running the normalizers

```

1 from policygraph import CONFIG, NORMALIZED
2 from policygraph.fetch import MANIFEST
3 from policygraph.normalize import acs, cdi_fair_plan, cdi_nonrenewal, documents, fhsz, zcta
4 from policygraph.util import load_yaml
5
6 NORMALIZERS = {m.__name__.rsplit(".", 1)[-1]: m for m in (zcta, fhsz, acs, cdi_nonrenewal,
7 cdi_fair_plan, documents)}
8
9 def run() -> None:
10     NORMALIZED.mkdir(parents=True, exist_ok=True)
11     cfg = load_yaml(CONFIG / "sources.yaml")["sources"]
12     manifest = load_yaml(MANIFEST)
13     (NORMALIZED / "documents.parquet").unlink(missing_ok=True) # document normalizers appe
14 nd, one source at a time
15     for norm in NORMALIZERS.values(): # order matters: zcta before fhsz
16         for name, spec in cfg.items():
17             if spec["normalizer"] == norm.__name__.rsplit(".", 1)[-1] and name in manifest:
18                 norm.run(spec, manifest[name])

```

- **6** A dictionary from normalizer name (the module's last dotted component, e.g. "zcta") to module, in the order they must run. ZCTA geometry must exist before the hazard overlay uses it.

- **13** The documents table is built by appending one source at a time (bulletins, then press releases), so it is deleted first to avoid stale rows from earlier runs.
- **14–17** For each normalizer, find every source whose `normalizer:` field names it and which was actually fetched, and call its `run(spec, files)`. Every normalizer has the same signature: the source's spec from `sources.yaml` and its list of fetched-file records.

3.3 `policygraph/normalize/zcta.py`: ZIP boundaries

```
5 CA_ZIPS = ("90000", "96200") # the cartographic file is national
8 def run(spec: dict, files: list[dict]) -> None:
9     g = gpd.read_file(f"zip://{files[0]['path']}").rename(columns={"ZCTA5CE20": "zip"})
10    g[g.zip.between(*CA_ZIPS)][["zip", "geometry"]].to_crs(3310).to_parquet(NORMALIZED / "zcta.parquet")
```

- **9** GeoPandas reads a shapefile straight out of the zip archive. The Census column `ZCTA5CE20` is renamed `zip`.
- **10** Keep only California ZIPs (90000–96199), only the two columns needed, reproject to California Albers (metres, area-true), and write parquet.

3.4 `policygraph/normalize/fhsz.py`: hazard share per ZIP

```
6 HAZ = {"Moderate": "moderate", "High": "high", "Very High": "very_high"} # LRA also carries NonWildland, dropped
7 HAZ_COL = "FHSZ_Description"
10 def overlap(zcta: gpd.GeoDataFrame, fhsz: gpd.GeoDataFrame) -> pd.DataFrame:
11     fhsz = fhsz.to_crs(zcta.crs).assign(haz=lambda d: d[HAZ_COL].map(HAZ)).dropna(subset=["haz"])[["haz", "geometry"]]
12     inter = gpd.overlay(zcta, fhsz, how="intersection", keep_geom_type=True)
13     inter["area"] = inter.area
14     share = inter.groupby(["zip", "haz"])["area"].sum().unstack(fill_value=0).reindex(zcta.zip, fill_value=0)
15     return share.div(zcta.set_index("zip").area, axis=0).reindex(columns=HAZ.values(), fill_value=0).reset_index()
```

`overlap` is a pure function (no file I/O), which is why it has a unit test.

- **11** Put the hazard polygons in the same CRS as the ZIPs, map CAL FIRE's labels to short names, drop classes we do not track (`NonWildland`), keep only the class and the shape.
- **12** `overlay(..., "intersection")` cuts every ZIP polygon by every hazard polygon, producing one piece per (ZIP, class) overlap.
- **13** Area of each piece in square metres.
- **14** Sum area by ZIP and class, pivot classes into columns, and make sure every ZIP appears even with zero hazard.
- **15** Divide by each ZIP's total area to get shares in 0–1, make sure all three class columns exist, and turn the index back into a `zip` column.


```

18 def run(spec: dict, files: list[dict]) -> None:
19     zcta = gpd.read_parquet(NORMALIZED / "zcta.parquet")
20     # ArcGIS GeoJSON omits the crs member; fetch requested outSR=3310
21     fhsz = gpd.GeoDataFrame(pd.concat(gpd.read_file(f["path"]).set_crs(3310, allow_override=True) for f in files))
22     overlap(zcta, fhsz).to_parquet(NORMALIZED / "hazard_share.parquet", index=False)

```

Reads the two fetched layers (state and local responsibility areas), stamps them with the CRS the fetch requested (GeoJSON from ArcGIS does not say), concatenates, overlays, writes.

3.5 policygraph/normalize/acs.py: Census medians

```

5  VARS = {
6      "B25077_001E": "median_home_value",
7      "B25035_001E": "median_year_built",
8      "B19013_001E": "median_income",
9  }
12 def parse(raw: pd.DataFrame) -> pd.DataFrame:
13     df = pd.DataFrame(raw.iloc[1:].values, columns=raw.iloc[0]).rename(columns=VARS | {"zip code tabulation area": "zip"})
14     vals = df[list(VARS.values())].astype(float)
15     return df[["zip"]].assign(**vals.mask(vals < 0)) # Census encodes suppressed cells as -666666666
18 def run(spec: dict, files: list[dict]) -> None:
19     parse(pd.read_json(files[0]["path"])).to_parquet(NORMALIZED / "acs.parquet", index=False)

```

- **5–9** The three ACS variable codes and their readable names.
- **13** The Census API returns a JSON array whose first row is the header. Rebuild a frame using row 0 as column names and rename.
- **14–15** Convert to numbers; the Census marks suppressed values with a large negative sentinel, which `mask(vals < 0)` turns into null.

3.6 policygraph/normalize/cdi_nonrenewal.py: the core counts

```

7  COMMON = {"County": "county", "ZIP Code": "zip", "Year": "year", "New": "new", "Renewed": "renewed"}
8  SPLIT = {"Insurer-Initiated Nonrenewed": "nonrenewed_insurer", "Insured-Initiated Nonrenewed": "nonrenewed_insured"}
9  TOTAL = {"Non-Renewed": "nonrenewed"} # the 2020–2023 release dropped the insurer/insured split
10 COUNTS = ["new", "renewed", "nonrenewed", "nonrenewed_insurer", "nonrenewed_insured"]
11 RELEASE_RE = re.compile(r"(\d{4}-\d{4})\.xlsx$")

```

CDI published two workbooks with different columns. The 2015–2021 release splits non-renewals by who initiated them (the insurer refusing versus the customer leaving); the 2020–2023 release gives only the total. `RELEASE_RE` pulls 2015–2021 or 2020–2023 from the file name.

```

14 def parse(df: pd.DataFrame, release: str) -> pd.DataFrame:
15     df = df.rename(columns=COMMON | SPLIT | TOTAL)
16     if missing := set(COMMON.values()) - set(df.columns):
17         raise ValueError(f"cdi_nonrenewal missing columns: {missing}")
18     if "nonrenewed" not in df.columns:
19         df["nonrenewed"] = df.nonrenewed_insurer.astype(int) + df.nonrenewed_insured.astype
(int)
20     out = df.reindex(columns=["county", "zip", "year", *COUNTS])
21     out[COUNTS] = out[COUNTS].apply(pd.to_numeric).astype("Int64")
22     county = out.county.str.strip()
23     return out.assign(
24         release=release,
25         county=county.mask(county == ""),
26         zip=out.zip.str.zfill(5),
27         year=out.year.astype(int),
28         policies=out.new + out.renewed + out.nonrenewed,
29     )

```

- **15** Rename whichever columns are present to the short names.
- **16–17** If any required column is absent, stop. This is what "schema drift fails loudly" looks like.
- **18–19** For the split release, total non-renewals is the sum of the two parts.
- **20–21** Select the columns in a fixed order (missing ones become null) and convert counts to nullable integers.
- **22–29** Add the release tag, blank county names become null (PO-box ZIPs have no county in the sheet), ZIPs are padded to five digits (the sheet drops leading zeros in some rows), year becomes int, and `policies` is the denominator used for every rate: new + renewed + non-renewed.

```

32 def run(spec: dict, files: list[dict]) -> None:
33     frames = [parse(pd.read_excel(f["path"], dtype=str), RELEASE_RE.search(f["url"]).group
(1)) for f in files]
34     pd.concat(frames, ignore_index=True).to_parquet(NORMALIZED / "cdi_nonrenewal.parquet",
index=False)

```

Read each workbook as text (so ZIPs keep their zeros), parse, stack, write. The two releases stay distinguishable by `release`.

3.7 policygraph/normalize/cdi_fair_plan.py: the FAIR Plan PDF

```

8 ROW_RE = re.compile(r"^\d{5}\b")
9 FY_RE = re.compile(r"9/30/(\d{4})")
10 def layout(text: str) -> tuple[list[int], int, list[int]]:
11     """Fiscal years from the footer; a row is zip, (growth, count) per year except the oldest, then the oldest count."""
12     years = list(dict.fromkeys(int(y) for y in FY_RE.findall(text)))
13     if not years:
14         raise ValueError("cdi_fair_plan: no fiscal-year footer found")
15     n = len(years)
16     return years, 2 * n, [*range(2, 2 * n, 2), 2 * n - 1]

```

The PDF is a table whose text extraction gives lines like 94549 91% 1,773 265% 927 76% 254 26% 144 114: the ZIP, then for each fiscal year except the oldest a growth percentage and a count, then

the oldest count alone. The years themselves are printed in a footer as 9/30/2025 9/30/2024

- **14** Collect the fiscal years in order, removing duplicates (the footer repeats on every page).
- **17–18** With `n` years a row has `2n` tokens; the counts sit at token positions 2, 4, ..., `2n-2` and the last position. Deriving this from the footer is what lets a future six-column release parse without a code change.

```
21 def parse(text: str) -> pd.DataFrame:
22     years, tokens, count_at = layout(text)
23     rows = []
24     for line in filter(ROW_RE.match, text.splitlines()):
25         tok = line.split()
26         if len(tok) != tokens:
27             raise ValueError(f"cdi_fair_plan: expected {tokens} tokens for {len(years)} fis
cal years, got {len(tok)}")
28         counts = [tok[i] for i in count_at]
29         rows += [{"zip": tok[0], "year": y, "fair_policies": c} for y, c in zip(years, coun
ts, strict=True) if c != "-"]
30     df = pd.DataFrame(rows)
31     return df.assign(fair_policies=df.fair_policies.str.replace(",", "").astype(int))
```

- **24** Only lines that start with five digits are data rows.
- **26–27** A row with the wrong token count means the layout changed; stop rather than mis-assign numbers.
- **28–29** Pick the counts, pair them with years, skip `-` (no policies that year).
- **31** Strip thousands separators and convert to integers.

```
34 def run(spec: dict, files: list[dict]) -> None:
35     text = "\n".join(p.extract_text() for p in PdfReader(files[0]["path"]).pages)
36     parse(text).to_parquet(NORMALIZED / "cdi_fair_plan.parquet", index=False)
```

pypdf extracts the text of every page (the `cryptography` dependency lets it open the encrypted file); the result goes through `parse`.

3.8 policygraph/normalize/html_text.py: press-release HTML to text

```

1  from html.parser import HTMLParser
3  BLOCK = {"p", "div", "li", "br", "h1", "h2", "h3", "h4", "ul", "ol", "tr", "hr"}
4  SKIP = {"script", "style"}
7  class _Text(HTMLParser):
8      def __init__(self) -> None:
9          super().__init__()
10         self.parts: list[str] = []
11         self.skip = 0
13     def handle_starttag(self, tag: str, attrs: list) -> None:
14         if tag in SKIP:
15             self.skip += 1
16         if tag in BLOCK:
17             self.parts.append("\n")
19     def handle_endtag(self, tag: str) -> None:
20         if tag in SKIP:
21             self.skip -= 1
22         if tag in BLOCK:
23             self.parts.append("\n")
25     def handle_data(self, data: str) -> None:
26         if not self.skip:
27             self.parts.append(data)

```

A tiny HTML-to-text converter built on Python's standard parser, so no new dependency.

- **3–4** Tags that start a new line, and tags whose content (JavaScript, CSS) must be dropped.
- **7–11** The parser keeps a list of text pieces and a counter of how deep we are inside skipped tags.
- **13–23** Entering or leaving a block tag inserts a line break; entering or leaving a skipped tag adjusts the counter.
- **25–27** Text is kept unless we are inside a skipped tag.

```

30 def to_text(html: str, start: str = "<H1>", end: str = 'class="content_right_column') -> str:
31     """CDI pages wrap the release in a left column between the headline and the sidebar; fail if either marker is missing."""
32     i, j = html.find(start), html.find(end)
33     if i < 0 or j < i:
34         raise ValueError(f"html_text: markers {start!r}..{end!r} not found")
35     p = _Text()
36     p.feed(html[i:j])
37     lines = (" ".join(line.split()) for line in "".join(p.parts).splitlines())
38     return "\n".join(line for line in lines if line)

```

- **32–34** CDI's page template puts the headline in `<H1>` and the sidebar in a `content_right_column` div. Take the text between them; if the template ever changes, stop loudly.
- **36–38** Feed that slice to the parser, join the pieces, collapse runs of whitespace within each line, and drop empty lines.

3.9 policygraph/normalize/documents.py: the documents table

```

12 DATE_RE = re.compile(r"(?:(:DATE|For Release):\s*|^)([A-Z][a-z]+ \d{1,2}, \d{4})(?:\s*[\—
-]|\s*$)", re.M)
15 def pdf_text(path: str) -> str:
16     return "\n".join(p.extract_text() or "" for p in PdfReader(path).pages)
19 def text_of(path: str, kind: str) -> str:
20     if kind == "pdf":
21         return pdf_text(path)
22     if kind == "html":
23         return to_text(Path(path).read_text(errors="replace"))
24     raise ValueError(f"documents: unsupported source type {kind!r}")
27 def published_at(text: str) -> date | None:
28     m = DATE_RE.search(text)
29     return pd.to_datetime(m.group(1)).date() if m else None

```

- **12** The publication date appears as `DATE: November 6, 2020` in bulletins, `For Release: May 13, 2025` in press releases, or as a line beginning `December 20, 2025` – in alerts. The pattern captures the `Month day, year` part in any of those positions.
- **15–24** Get the text of a PDF or an HTML file; any other type is an error.
- **27–29** Find the first date match and convert it to a `date`; `None` if there is none.

```

32 def run(spec: dict, files: list[dict]) -> None:
33     """doc_id hashes the extracted text: CDI's HTML carries per-request tokens, so raw byte
s never repeat."""
34     texts = [text_of(f["path"], spec["type"]) for f in files]
35     df = pd.DataFrame(
36         [
37             {
38                 "doc_id": sha256(t.encode()),
39                 "source": spec["doc_source"],
40                 "url": f["url"],
41                 "published_at": published_at(t),
42                 "text": t,
43             }
44             for f, t in zip(files, texts, strict=True)
45         ]
46     )
47     out = NORMALIZED / "documents.parquet"
48     if out.exists():
49         df = pd.concat([pd.read_parquet(out), df]).drop_duplicates("doc_id")
50     df.to_parquet(out, index=False)

```

- **38** The document id is the hash of the `text`, not the downloaded bytes. CDI's HTML embeds a fresh session token on every download, so hashing bytes would give a new id every time; the extracted text is stable.
- **39** `doc_source` (`cdi_bulletin` or `cdi_press`) comes from `sources.yaml`.
- **47–50** Append to the existing table (the bulletins normalizer runs, then the press one), removing duplicate ids.

3.10 `policygraph/extract.py`: submitting documents to the service

```

10 SERVICE = os.environ.get("EXTRACTION_SERVICE_URL", "http://localhost:8000")
13 def run(schema_v: str = "1.0", force: bool = False) -> None:
14     """force re-runs the model even for documents whose extraction id already has an artifact."""
15     docs = pd.read_parquet(NORMALIZED / "documents.parquet")
16     with httpx.Client(base_url=SERVICE, timeout=60) as c:
17         run_id = c.post("/v1/runs", json={"schema_v": schema_v}).json()["run_id"]
18         for d in docs.iteruples():
19             body = {"run_id": run_id, "doc_id": d.doc_id, "source": d.source, "text": d.text, "force": force}
20             c.post("/v1/jobs", json=body).raise_for_status()
21             while (m := c.get(f"/v1/manifests/{run_id}").json()["status"]) != "complete":
22                 time.sleep(2)
23             dump_yaml(CONFIG / "extraction_manifest.yaml", m)
24             if m["failed"] or m["errors"]:
25                 raise RuntimeError(f"extraction run {run_id}: failed={m['failed']} errors={m['errors']}")

```

The pipeline never calls a model. It talks to the service over HTTP.

- **17** Create a run; the service replies with an id.
- **18–20** Submit one job per document, carrying the text and the `force` flag.
- **21–22** Poll the run's manifest every two seconds until every job is done.
- **23** Save the manifest. This file, committed to git, is what pins exactly which extraction artifacts the graph loads.
- **24–25** If any document failed validation or errored, the stage fails.

3.11 policygraph/resolve.py: mentions to entity ids

The model emits *mentions* ("Eaton Fire 2025", "State Farm", "94563"). Resolution turns each into a canonical entity id, cheapest method first.

```

13 SERVICE = os.environ.get("EXTRACTION_SERVICE_URL", "http://localhost:8000")
14 PINNED = CONFIG / "resolutions.yaml" # every answer the service gave, committed so `make resolve` replays offline
15 ZIP_RE = re.compile(r"^\d{5}$")
16 BILL_RE = re.compile(r"^(SB|AB)\s?(\\d+)$", re.I)
17 FIRE_RE = re.compile(r"^(.+?) Fire (\\d{4})$", re.I)
18 MORATORIUM_RE = re.compile(r"^(.+?) Fire moratorium (\\d{4}-\\d{2}-\\d{2})$", re.I)
19 Ask = Callable[[list[dict], list[dict]], list[dict]]
22 @dataclass(frozen=True)
23 class Resolved:
24     mention: str
25     type: str
26     entity_id: str
27     method: str
28     confidence: float
31 def slug(s: str) -> str:
32     return re.sub(r"^[a-z0-9]+", "-", s.lower()).strip("-")

```

- **15–18** The four canonical patterns: five digits; SB 824 / AB2996; <Name> Fire <year>; <Name> Fire moratorium <date>. The extraction prompt tells the model to write mentions in exactly these shapes.

- **19** The type of "a function that asks the service": takes mentions and candidates, returns answers.
- **22–28** One resolution record.
- **31–32** `slug("SCU Lightning Complex") → scu-lightning-complex`: lowercase, non-alphanumerics to hyphens.

```

35 def canonical(mention: str, etype: str) -> Resolved | None:
36     """The returned mention is the original string: the graph looks edges up by the artifact's exact mention."""
37     m = mention.strip()
38     if etype == "ZIP" and ZIP_RE.match(m):
39         return Resolved(mention, etype, f"zip:{m}", "canonical", 1.0)
40     if etype == "Regulation" and (b := BILL_RE.match(m)):
41         return Resolved(mention, etype, f"bill:ca:{b.group(1).upper()}{b.group(2)}", "canonical", 1.0)
42     if etype == "Fire" and (f := FIRE_RE.match(m)):
43         return Resolved(mention, etype, f"fire:{slug(f.group(1))}:{f.group(2)}", "canonical", 1.0)
44     if etype == "Moratorium" and (mo := MORATORIUM_RE.match(m)):
45         return Resolved(mention, etype, f"moratorium:{mo.group(2)}:{slug(mo.group(1))}", "canonical", 1.0)
46     return None

```

Step 1. Each branch checks the mention's declared type *and* its shape, and mints the id. Type checking prevents the classic failure of an insurer resolving to an official. The confidence is 1.0 because these are rules, not guesses. Note `mention` (original) is returned, not `m` (stripped): the graph stage looks edges up by the exact string the model used.

```

49 def alias_table(entities: list[dict]) -> dict[tuple[str, str], str]:
50     return {(a.lower(), e["type"]): e["id"] for e in entities for a in (e["name"], *e["aliases"])}
53 def alias(mention: str, etype: str, aliases: dict[tuple[str, str], str]) -> Resolved | None:
54     eid = aliases.get((mention.strip().lower(), etype))
55     return Resolved(mention, etype, eid, "alias", 0.98) if eid else None
58 def pending(mention: str, etype: str) -> Resolved:
59     return Resolved(mention, etype, f"pending:{etype.lower()}:{slug(mention)}", "unresolved", 0.0)

```

Step 2. `alias_table` flattens `config/entities.yaml` into `(lowercase name or alias, type) → id`. `alias` looks a mention up exactly. `pending` mints a placeholder id for anything nobody could resolve, so the graph still loads and the gap is visible.

```

62 def from_pin(pin: dict) -> Resolved:
63     if pin["entity_id"] is None:
64         return pending(pin["mention"], pin["type"])
65     return Resolved(pin["mention"], pin["type"], pin["entity_id"], pin["method"], pin["confidence"])
68 def ask_service(mentions: list[dict], candidates: list[dict]) -> list[dict]:
69     with httpx.Client(base_url=SERVICE, timeout=600) as c:
70         r = c.post("/v1/resolve", json={"mentions": mentions, "candidates": candidates})
71         r.raise_for_status()
72         return r.json()

```

- **62–65** Turn a pinned service answer (from `config/resolutions.yaml`) into a `Resolved`; an answer of "nobody" becomes pending.
- **68–72** Steps 3–4 live in the service; this posts the residual mentions and the candidate entities and returns its answers.

```

75 def resolve_all(mentions: pd.DataFrame, entities: list[dict], pins: dict[tuple[str, str], dict], ask: Ask | None) -> pd.DataFrame:
76     """Cheapest first: canonical key, alias table, pinned service answer, then the service (kNN + judge), else pending."""
77     aliases, out, residual = alias_table(entities), {}, []
78     for m in mentions.drop_duplicates(["mention", "type"]).itertuples():
79         key = (m.mention, m.type)
80         if r := canonical(*key) or alias(*key, aliases):
81             out[key] = r
82         elif key in pins:
83             out[key] = from_pin(pins[key])
84         else:
85             residual.append({"mention": m.mention, "type": m.type, "context": m.context})
86     if residual and ask is not None:
87         cands = [{"entity_id": e["id"], "type": e["type"], "name": e["name"], "description": e["description"]} for e in entities]
88         for pin in ask(residual, cands):
89             pins[(pin["mention"], pin["type"])] = pin
90             out[(pin["mention"], pin["type"])] = from_pin(pin)
91     for r in residual:
92         out.setdefault((r["mention"], r["type"]), pending(r["mention"], r["type"]))
93     return pd.DataFrame([asdict(r) for r in out.values()])

```

The whole ladder in one function.

- **78–85** For each distinct (mention, type): try the rules, then the alias table, then a previously pinned answer; anything left is "residual" and carries its context (a quoted span) for the judge.
- **86–90** If there is a residual and we are online, ask the service once for all of them; record every answer in `pins` (so it gets committed) and in the output.
- **91–92** Anything still unanswered (offline, or the service said "nobody") becomes pending.
- **93** Return a table.

```

96 def mentions_in(artifact: dict) -> pd.DataFrame:
97     ctx = {}
98     for e in artifact["edges"]:
99         ctx.setdefault(e["src_mention"], e["span"])
100         ctx.setdefault(e["dst_mention"], e["span"])
101     return pd.DataFrame(
102         [{"mention": e["mention"], "type": e["type"], "context": ctx.get(e["mention"], "")}
103         for e in artifact["entities"]]
104     )

```

From one extraction artifact, list its entities with, as context, the first quoted span of any edge that touches the mention. The judge reads that context.


```

106 def run(force: bool = False) -> None:
107     manifest = load_yaml(CONFIG / "extraction_manifest.yaml")
108     entities = load_yaml(CONFIG / "entities.yaml")["entities"]
109     pins = {} if force else {(p["mention"], p["type"]): p for p in (load_yaml(PINNED) if PINNED.exists()) else []}
110     arts = [json.loads((EXTRACTIONS / f"{x}.json").read_text()) for x in manifest["extractions"]]
111     mentions = pd.concat([mentions_in(a) for a in arts], ignore_index=True)
112     ask = ask_service if os.environ.get("EXTRACTION_SERVICE_URL") else None
113     df = resolve_all(mentions, entities, pins, ask)
114     seen = set(zip(mentions.mention, mentions.type, strict=True))
115     dump_yaml(
116         PINNED, sorted((p for p in pins.values() if (p["mention"], p["type"]) in seen), key=
117         =lambda p: (p["type"], p["mention"]))
118     )
118     df.to_parquet(EXTRACTIONS.parent / "normalized" / "resolved.parquet", index=False)
119     print(df.method.value_counts().to_dict())

```

- **107–111** Load the pinned manifest, the entity registry, the previous pins (dropped under `force`), and every mention in every pinned artifact.
- **112** Only ask the service if its URL is configured; otherwise replay offline.
- **114–117** Write the pins back, keeping only mentions that still exist, sorted for a stable diff.
- **118–119** Save the resolution table and print a count per method.

3.12 `policygraph/names.py`: readable names for rule-minted ids

```

3  FIRE_RE = re.compile(r"^fire:([a-z0-9-]+):(\d{4})$")
4  MOR_RE = re.compile(r"^moratorium:(\d{4}-\d{2}-\d{2}):([a-z0-9-]+)$")
5  FIXED = {
6      "market:ca:voluntary": "California voluntary market (all admitted insurers)",
7      "fhsz:very_high": "FHSZ Very High",
8      "fhsz:high": "FHSZ High",
9      "fhsz:moderate": "FHSZ Moderate",
10 }
11
12 def title(slug: str) -> str:
13     return " ".join(w.upper() if w in {"scu", "lnu", "czu", "shf", "sqf"} else w.capitalize() for w in slug.split("-"))
14
15 def humanize(entity_id: str, places: dict[str, str]) -> str:
16     """Readable name for ids that are minted by rule and so have no entry in config/entities.yaml."""
17
18     if entity_id in FIXED:
19         return FIXED[entity_id]
20     if entity_id.startswith("zip:"):
21         z = entity_id[4:]
22         return f"{places[z]} {z}" if z in places else z
23     if entity_id.startswith("bill:ca:"):
24         return re.sub(r"^(?=[A-Z])(\d+)$", r"\1 \2", entity_id[8:])
25     if m := FIRE_RE.match(entity_id):
26         return f"{title(m.group(1))} Fire {m.group(2)}"
27     if m := MOR_RE.match(entity_id):
28         return f"{title(m.group(2))} Fire moratorium {m.group(1)}"
29     if entity_id.startswith("pending:"):
30         return entity_id.split(":", 2)[2].replace("-", " ") + " (unresolved)"
31     return entity_id

```

Entities made by rule (ZIPs, fires, moratoria, bills) have no hand-written name, so this derives one:
`zip:94549` → Lafayette 94549 (from `config/places.yaml`), `fire:scu-lightning-complex:2020` →

SCU Lightning Complex Fire 2020 (fire-complex acronyms stay upper-case), bill:ca:SB824 → SB 824. The app's timeline and explorer show these.

3.13 policygraph/graph.py: building the DuckDB graph

```
11 COLS = ["src", "rel", "dst", "valid_from", "valid_to", "props", "doc_id", "span", "extractor", "confidence"]
12 MARKET = "market:ca:voluntary" # CDI publishes ZIP totals only; no insurer identity is in the public data
13 MORATORIUM_RE = re.compile(r"^moratorium:\d{4}-\d{2}-\d{2}$:")
14 ACS = ["median_home_value", "median_year_built", "median_income"]
17 def _year(s: pd.Series) -> pd.Series:
18     return pd.to_datetime(s.astype(str) + "-01-01")
21 def _props(df: pd.DataFrame) -> list[str]:
22     return [json.dumps({k: (None if pd.isna(v) else v) for k, v in r.items()}) for r in df.to_dict("records")]
```

- **11** The edge columns, in the order the schema expects.
- **12** Because CDI's counts name no insurer, every count edge starts at one anonymous "voluntary market" node.
- **17–18** A year becomes the date January 1 of that year (edges need dates).
- **21–22** Turn each row of a frame into a JSON string, with nulls as JSON `null`.

```

25 def deterministic_edges(nr: pd.DataFrame, haz: pd.DataFrame, fair: pd.DataFrame) -> pd.DataFrame:
26     haz_long = haz.melt("zip", var_name="haz", value_name="pct")
27     nr_props = nr[["nonrenewed", "nonrenewed_insurer", "nonrenewed_insured", "new", "renewed", "policies", "release"]]
28     frames = [
29         pd.DataFrame(
30             {
31                 "src": MARKET,
32                 "rel": "NONRENEWED_IN",
33                 "dst": "zip:" + nr.zip,
34                 "valid_from": _year(nr.year),
35                 "props": _props(nr_props.rename(columns={"nonrenewed": "count"})),
36                 "extractor": "deterministic:cdi_nonrenewal:" + nr.release, # releases over
lap 2020-21 and disagree
37             }
38         ),
39         pd.DataFrame(
40             {
41                 "src": "zip:" + haz_long.zip,
42                 "rel": "HAZARD_SHARE",
43                 "dst": "fhsz:" + haz_long.haz,
44                 "props": _props(haz_long[["pct"]]),
45                 "extractor": "deterministic:fhsz_v1",
46             }
47         ),
48         pd.DataFrame(
49             {
50                 "src": "zip:" + fair.zip,
51                 "rel": "HAS_FAIR_PLAN_POLICIES",
52                 "dst": "insurer:fairplan",
53                 "valid_from": _year(fair.year),
54                 "props": _props(fair[["fair_policies"]].rename(columns={"fair_policies": "count"})),
55                 "extractor": "deterministic:cdi_fair_v1",
56             }
57         ),
58     ]
59     return pd.concat(frames, ignore_index=True).assign(confidence=1.0).reindex(columns=COLS)

```

Three edge families from the three tabular sources.

- **26** The hazard table has one column per class; `melt` turns it into (zip, class, share) rows.
- **29–38** One `NONRENEWED_IN` edge per ZIP-year-release. The counts travel in `props`; the extractor tag includes the release so the two disagreeing releases are never confused.
- **39–47** One `HAZARD_SHARE` edge per ZIP and class.
- **48–57** One `HAS_FAIR_PLAN_POLICIES` edge per ZIP and fiscal year.
- **59** Stack them, mark confidence 1.0 (deterministic), and align to the edge columns (absent ones become null).

```

62 def window(edge: dict, dst: str) -> tuple[str | None, str | None]:
63     """A moratorium runs one year from the declaration date (Ins. Code 675.1(b)(1)); the date is in its canonical id."""
64     if edge["valid_from"] or not (m := MORATORIUM_RE.match(dst)):
65         return edge["valid_from"], edge["valid_to"]
66     start = pd.Timestamp(m.group(1))
67     return str(start.date()), str((start + pd.DateOffset(years=1)).date())

```

If the model gave an edge no dates but its target is a moratorium, the id contains the declaration date, and the law says the moratorium lasts one year: fill both dates from that.

```
70 def llm_edges(manifest: dict, resolved: dict[str, str]) -> pd.DataFrame:
71     rows = []
72     for xid in manifest["extractions"]:
73         art = json.loads((EXTRACTIONS / f"{xid}.json").read_text())
74         rows += [
75             {
76                 "src": resolved[e["src_mention"]],
77                 "rel": e["rel"],
78                 "dst": resolved[e["dst_mention"]],
79                 "valid_from": window(e, resolved[e["dst_mention"]])[0],
80                 "valid_to": window(e, resolved[e["dst_mention"]])[1],
81                 "props": json.dumps(e["props"]),
82                 "doc_id": art["doc_id"],
83                 "span": e["span"],
84                 "extractor": f"llm:{art['model_id']}" + ("|zip_block" if e["props"].get("expanded_from") == "zip_block" else ""),
85                 "confidence": e["confidence"],
86             }
87             for e in art["edges"]
88         ]
89     return pd.DataFrame(rows, columns=COLS)
```

For every pinned artifact, every edge becomes a graph edge: mentions are swapped for resolved ids (`resolved[...]` raises if a mention was never resolved, on purpose), dates are filled by `window`, and provenance is copied through. Edges the service expanded deterministically from ZIP-list blocks get `|zip_block` appended to the extractor tag so they can be told apart from what the model wrote itself.

```
92 def zip_attrs(nr: pd.DataFrame, acs: pd.DataFrame) -> dict[str, dict]:
93     """County from CDI (blank for PO-box ZIPs) and ACS medians, keyed by ZIP; suppressed ACS cells stay null."""
94     county = nr.dropna(subset=["county"]).drop_duplicates("zip").set_index("zip").county
95     df = acs.set_index("zip")[ACS].join(county, how="outer")
96     return {z: {k: (None if pd.isna(v) else v) for k, v in r.items()} for z, r in df.to_dict("index").items()}
97
98 def entities_from(edges: pd.DataFrame, attrs: dict[str, dict], names: dict[str, str], places: dict[str, str]) -> pd.DataFrame:
99     ids = pd.Series(pd.unique(pd.concat([edges.src, edges.dst])))
100     return pd.DataFrame(
101         {
102             "entity_id": ids,
103             "type": ids.str.split(":").str[0],
104             "canonical_name": [names.get(i) or humanize(i, places) for i in ids],
105             "attrs": [json.dumps(attrs[i[4:]]) if i.startswith("zip:") and i[4:] in attrs else None for i in ids],
106         }
107     )
108
```

- **92–96** Per ZIP: its county (from CDI) and its three ACS medians, merged with an outer join so a ZIP present in either source appears.
- **99–108** The entity table is derived from the edges: every id that appears as a source or target. Type is the id's prefix; the name comes from `entities.yaml` if listed, else from `humanize`; ZIPs get their attributes as JSON.

```

111 def run() -> None:
112     nr, haz, fair, acs, docs = (
113         pd.read_parquet(NORMALIZED / f"{n}.parquet")
114         for n in ("cdi_nonrenewal", "hazard_share", "cdi_fair_plan", "acs", "documents")
115     )
116     resolved = pd.read_parquet(NORMALIZED / "resolved.parquet").set_index("mention").entity_id.to_dict()
117     names = {e["id"]: e["name"] for e in load_yaml(CONFIG / "entities.yaml")["entities"]}
118     edges = pd.concat([deterministic_edges(nr, haz, fair), llm_edges(load_yaml(CONFIG / "extraction_manifest.yaml"), resolved)])
119     key = edges[["src", "rel", "dst", "valid_from", "doc_id", "extractor"]]
120     edges["edge_id"] = [stable_id(*r) for r in key.itertuples(index=False)]
121     ents = entities_from(edges, zip_attrs(nr, acs), names, load_yaml(CONFIG / "places.yaml"))
122     GRAPH_DB.unlink(missing_ok=True) # the file is derived from the pinned inputs; history lives in those, not here
123     with duckdb.connect(str(GRAPH_DB)) as con:
124         con.execute((CONFIG / "schema.sql").read_text())
125         for table, df in (("documents", docs), ("entities", ents), ("edges", edges)):
126             con.register(f"new_{table}", df) # a bare DataFrame name would resolve to the table itself
127         con.execute(f"INSERT OR IGNORE INTO {table} BY NAME SELECT * FROM new_{table}")

```

- **112–117** Load the five normalized tables, the resolution map (mention → id), and the hand-written names.
- **118** All edges: deterministic plus model-derived.
- **119–120** Each edge id is a hash of what makes it unique: endpoints, relation, start date, document and extractor. The same fact always gets the same id, so re-running cannot duplicate it.
- **121** Build the entity table.
- **122** Delete the old database file: it is entirely derived from the pinned inputs (the "history" the design promises lives in those inputs, which are append-only), so rebuilding from scratch is the reproducible choice.
- **123–127** Create the tables from `schema.sql`, register each DataFrame under a temporary name (DuckDB can query pandas frames as if they were tables), and insert. `INSERT OR IGNORE` skips any id that already exists.

Part 4. Insights and the app

Every insight module has one public function, `compute(con) -> DataFrame`, that takes a read-only DuckDB connection and returns a table. `policygraph/insights/__init__.py` runs them all and writes one parquet file per module into `data/insights/`. The app reads those files and nothing else.

4.1 `policygraph/insights/__init__.py`

```
17 MODULES = (  
18     hazard_residual,  
19     moratorium_deferral,  
20     regulatory_alignment,  
21     fair_mirror,  
22     actor_centrality,  
23     evidence,  
24     events,  
25     trend,  
26     explorer,  
27 )  
30 def run() -> None:  
31     INSIGHTS.mkdir(parents=True, exist_ok=True)  
32     with duckdb.connect(str(GRAPH_DB), read_only=True) as con:  
33         for m in MODULES:  
34             m.compute(con).to_parquet(INSIGHTS / f"{m.__name__.rsplit('.', 1)[-1]}.parque  
35             t", index=False)  
36             geometry.write(con, INSIGHTS / "cc_zips.geojson")
```

Five findings modules, then four "serving" tables (evidence, events, trend, explorer) and the county geometry file. Each parquet is named after its module.

4.2 `hazard_residual.py`: finding 1

```
7 FITS = (  
8     (2021, "2015-2021", "nonrenewed_insurer"),  
9     (2023, "2020-2023", "count"),  
10 )  
12 SPECS = {"hazard": ["very_high", "high"], "hazard+acs": ["very_high", "high", "log_home_val  
ue", "log_income"]}
```

Two fits: the last year with the insurer/insured split (2021, insurer-initiated counts) and the last year of data at all (2023, total counts). Two specifications each: hazard alone, and hazard plus log home value and log income (the "is it price, not fire?" test).

```

14 SQL = """
15 WITH nr AS (
16     SELECT dst AS zip, json_extract(props, ?)::INT AS n, json_extract(props, '$.policies')::IN
T AS p
17     FROM edges WHERE rel='NONRENEWED_IN' AND year(valid_from)=? AND json_extract_string(prop
s, '$.release')=?),
18     haz AS (
19         SELECT src AS zip,
20             max(CASE WHEN dst='fhsz:very_high' THEN json_extract(props, '$.pct')::DOUBLE END) A
S very_high,
21             max(CASE WHEN dst='fhsz:high' THEN json_extract(props, '$.pct')::DOUBLE END) AS hig
h
22     FROM edges WHERE rel='HAZARD_SHARE' GROUP BY 1)
23     SELECT substr(nr.zip, 5) AS zip, json_extract_string(e.attrs, '$.county') AS county, sum
(n)::DOUBLE/sum(p) AS rate,
24         coalesce(any_value(haz.very_high),0) AS very_high, coalesce(any_value(haz.high),0) A
S high, sum(p) AS policies,
25         any_value(json_extract(e.attrs, '$.median_home_value')::DOUBLE) AS home_value,
26         any_value(json_extract(e.attrs, '$.median_income')::DOUBLE) AS income
27 FROM nr LEFT JOIN haz USING (zip) JOIN entities e ON e.entity_id = nr.zip
28 GROUP BY 1,2 HAVING sum(p) > 50 AND sum(n) IS NOT NULL
29 """

```

SQL with three ? placeholders (the measure's JSON path, the year, the release).

- **15–17** nr: for the chosen year and release, each ZIP's non-renewal count `n` (insurer-initiated or total) and policies `p`, pulled out of the edge's JSON props.
- **18–22** haz: pivot each ZIP's Very High and High area shares into columns.
- **23–28** Join, strip the `zip:` prefix, compute the rate, default missing hazard to 0, pull county and ACS medians from the entity attributes, and drop tiny ZIPs (≤ 50 policies) whose rates are noise.

```

32 def fit(con: duckdb.DuckDBPyConnection, year: int, release: str, measure: str) -> pd.DataFrame:
33     base = con.execute(SQL, [f"${measure}", year, release]).df()
34     base["log_home_value"], base["log_income"] = np.log(base.home_value), np.log(base.income)
35     out = []
36     for spec, cols in SPECS.items():
37         df = base.dropna(subset=cols).copy()
38         x = sm.add_constant(df[cols])
39         ols = sm.OLS(df.rate, x).fit()
40         df["predicted"] = ols.predict(x)
41         df["residual"] = df.rate - df.predicted
42         coefs = {f"coef_{c}": ols.params[c] for c in cols} | {f"p_{c}": ols.pvalues[c] for
c in cols}
43         out.append(df.assign(year=year, release=release, measure=measure, spec=spec, r2=ols.rsquared, n=len(df), **coefs))
44     return pd.concat(out).sort_values(["spec", "residual"], ascending=[True, False])
47 def compute(con: duckdb.DuckDBPyConnection) -> pd.DataFrame:
48     return pd.concat([fit(con, *f) for f in FITS], ignore_index=True)

```

- **33–34** Run the query and add log-transformed price columns (log because a doubling should matter the same at any level).
- **36–43** For each spec: drop ZIPs missing a regressor, add the intercept column, fit ordinary least squares, compute each ZIP's predicted rate and residual, and attach the fit's coefficients, p-values, R^2 and sample size to every row so the app can show them.

- **44** Sort so the biggest positive residuals (most above the line) come first; the row number becomes the ZIP's statewide rank.

4.3 `matching.py`: the matched control

```

5 def nearest(treated: pd.DataFrame, pool: pd.DataFrame, cols: list[str], k: int = 5) -> list
[str]:
6     """Union of each treated row's k nearest pool rows in standardized feature space; index
values are the ids."""
7     both = pd.concat([treated[cols], pool[cols]])
8     mu, sd = both.mean(), both.std().replace(0, 1)
9     t = ((treated[cols] - mu) / sd).to_numpy(dtype=float)
10    p = ((pool[cols] - mu) / sd).to_numpy(dtype=float)
11    d = np.linalg.norm(t[:, None, :] - p[None, :, :], axis=2)
12    picks = np.argsort(d, axis=1)[:, : min(k, len(pool))]
13    return sorted({pool.index[i] for row in picks for i in row})

```

Given treated rows (protected ZIPs) and a pool (unprotected ZIPs) with the same feature columns:

- **7–10** Standardise each feature (subtract the mean, divide by the standard deviation) so hazard share and log policy count are on the same scale; a zero deviation is replaced by 1 to avoid dividing by zero.
- **11** Euclidean distance from every treated row to every pool row (a treated × pool matrix, via numpy broadcasting).
- **12** For each treated row, the indices of its `k` closest pool rows.
- **13** The union of those, as sorted ids.

4.4 `moratorium_deferral.py`: finding 3

```

9  FITS = (("2015–2021", "nonrenewed_insurer"), ("2020–2023", "count"))
10 MATCH_ON = ["very_high", "log_policies"]
12 PROT = "SELECT DISTINCT src AS zip, dst AS moratorium, valid_from, valid_to FROM edges WHERE
rel='PROTECTED_BY'"
13 NR = """
14 SELECT dst AS zip, year(valid_from) AS y, json_extract(props, ?)::INT AS n, json_extract(pr
ops, '$.policies')::INT AS p
15 FROM edges WHERE rel='NONRENEWED_IN' AND json_extract_string(props, '$.release')=?
16 """
17 HAZ = """
18 SELECT src AS zip, json_extract(props, '$.pct')::DOUBLE AS very_high FROM edges WHERE rel='H
AZARD_SHARE' AND dst='fhsz:very_high'
19 """
22 def moratorium_year(valid_from: pd.Series, valid_to: pd.Series) -> pd.Series:
23     start, end = pd.to_datetime(valid_from), pd.to_datetime(valid_to).fillna(pd.to_datetime
(valid_from))
24     return (start + (end - start) / 2).dt.year

```

- **9** Each release is analysed with the measure it publishes.
- **12–19** Three queries: who was protected by which moratorium and when; the per-ZIP-year counts for a release; each ZIP's Very High share.
- **22–24** A moratorium's "year" is the calendar year containing the midpoint of its 12-month window. A declaration on 27 October 2019 runs to 27 October 2020; its midpoint is April

2020, so 2020 is "during", 2019 "before", 2021 "after".

```
27 def windows(nr: pd.DataFrame, y0: int) -> pd.DataFrame:
28     w = nr[nr.y.between(y0 - 1, y0 + 1)].pivot_table(index="zip", columns="y", values="n",
aggfunc="sum")
29     return w.reindex(columns=[y0 - 1, y0, y0 + 1]).set_axis(["before", "during", "after"],
axis=1)
32 def features(nr: pd.DataFrame, haz: pd.DataFrame, y: int) -> pd.DataFrame:
33     p = nr[nr.y == y].groupby("zip").p.sum()
34     return pd.DataFrame({"log_policies": np.log1p(p)}).join(haz.set_index("zip")).fillna
({"very_high": 0})
37 def summarise(grp: str, label: str, w: pd.DataFrame) -> dict:
38     s = w.sum(min_count=1)
39     return {"grp": grp, "zip": label, "n_zips": len(w), "before": s.before, "during": s.dur
ing, "after": s.after}
```

- **27–29** Pivot the counts of the three years around `y0` into before / during / after columns, one row per ZIP (a year absent from the release becomes null).
- **32–34** Matching features as of the year before: `log(1 + policies)` and Very High share.
- **37–39** Sum a group of ZIPs into one row (`min_count=1` keeps the sum null if every value is null).

```
42 def rows(prot: pd.DataFrame, nr: pd.DataFrame, haz: pd.DataFrame) -> list[dict]:
43     out, everywhere = [], set(prot.zip)
44     for (mor, y0), g in prot.groupby(["moratorium", "y0"]):
45         w = windows(nr, y0)
46         treated = w.index.intersection(set(g.zip))
47         if not len(treated) or w.loc[treated].isna().all().any():
48             continue # the window is outside this release
49         pool = w.index.difference(everywhere)
50         feats = features(nr, haz, y0 - 1)
51         matched = nearest(feats.reindex(treated).fillna(0), feats.reindex(pool).fillna(0),
MATCH_ON)
52         out += [
53             {"grp": "protected", "zip": z[4:], "n_zips": 1, **w.loc[z].to_dict()} | {"morat
orium": mor, "y0": y0} for z in treated
54         ]
55         out += [
56             summarise("control_all", "all other CA ZIPs", w.loc[pool]) | {"moratorium": mo
r, "y0": y0},
57             summarise("control_matched", "matched CA ZIPs", w.loc[matched]) | {"moratoriu
m": mor, "y0": y0},
58         ]
59     return out
```

- **43** `everywhere` is every ZIP protected by *any* moratorium; none of them may serve as a control.
- **44–48** For each moratorium: build the window, find the protected ZIPs that have data, and skip the moratorium if a whole column is missing (the window falls outside this release).
- **49–51** The control pool is every ZIP never protected; the matched control is the nearest neighbours by hazard share and policy count.
- **52–58** Emit one row per protected ZIP and one summary row for each control group.

```

62 def compute(con: duckdb.DuckDBPyConnection) -> pd.DataFrame:
63     prot = con.execute(PROT).df()
64     prot["y0"] = moratorium_year(prot.valid_from, prot.valid_to)
65     haz = con.execute(HAZ).df()
66     frames = []
67     for release, measure in FITS:
68         nr = con.execute(NR, [f"${measure}", release]).df()
69         frames.append(pd.DataFrame(rows(prot, nr, haz)).assign(release=release, measure=measure))
70     df = pd.concat(frames, ignore_index=True)
71     before = df["before"].replace(0, np.nan)
72     return df.assign(during_ratio=df["during"] / before, after_ratio=df["after"] / before)

```

Run `rows` for each release and add the two ratios the write-up quotes (a zero "before" is treated as missing rather than dividing by it).

4.5 `events.py`, `trend.py`, `regulatory_alignment.py`: finding 4

`events.py` lists every dated thing the documents contained:

```

6  SQL = """
7  SELECT CASE WHEN rel='TRIGGERED' THEN 'fire' WHEN rel='ISSUED' AND dst LIKE 'moratorium:%'
  THEN 'moratorium'
8             WHEN rel IN ('ISSUED','AUTHORED') THEN 'regulation' WHEN rel='FILED' THEN 'rate
  _filing'
9             ELSE 'rate_decision' END AS kind,
10     rel, e.src, e.dst, coalesce(s.canonical_name, e.src) AS src_name, coalesce(t.canonical_name, e.dst) AS dst_name,
11     e.valid_from AS date, e.valid_to, e.props, e.span, e.confidence, d.url,
12     (SELECT count(*) FROM edges p JOIN entities z ON z.entity_id = p.src
13      WHERE p.rel='PROTECTED_BY' AND p.dst = e.dst AND json_extract_string(z.attrs,'$.county') = ?) AS cc_zips
14 FROM edges e JOIN documents d USING (doc_id)
15 LEFT JOIN entities s ON s.entity_id = e.src LEFT JOIN entities t ON t.entity_id = e.dst
16 WHERE e.rel IN ('TRIGGERED','ISSUED','FILED','DECIDED','AUTHORED') AND e.valid_from IS NOT
  NULL
17 ORDER BY e.valid_from, kind, e.dst
18 """

```

Each dated document edge becomes an event with a `kind` (fire, moratorium, regulation, rate filing, rate decision), readable names, its span and URL, and, for moratoria, how many Contra Costa ZIPs it protected (the sub-query on lines 12–13).

`trend.py` computes the county and statewide series:

```

5  SQL = """
6  SELECT json_extract_string(props,'$.release') AS release, year(valid_from) AS y,
7          CASE WHEN json_extract_string(e.attrs,'$.county') = ? THEN ? ELSE 'California' END AS scope,
8          sum(json_extract(props,'$.count')::INT) AS nonrenewed,
9          sum(json_extract(props,'$.nonrenewed_insurer')::INT) AS nonrenewed_insurer,
10         sum(json_extract(props,'$.policies')::INT) AS policies
11 FROM edges JOIN entities e ON e.entity_id = dst WHERE rel='NONRENEWED_IN' GROUP BY 1,2,3
12 """
15 FAIR = """
16 SELECT 'fair-plan-fy' AS release, year(valid_from) AS y,
17         CASE WHEN json_extract_string(e.attrs,'$.county') = ? THEN ? ELSE 'California' END AS scope,
18         sum(json_extract(props,'$.count')::INT) AS fair
19 FROM edges JOIN entities e ON e.entity_id = src WHERE rel='HAS_FAIR_PLAN_POLICIES' GROUP BY 1,2,3
20 """
23 def fair_series(con: duckdb.DuckDBPyConnection, county: str) -> pd.DataFrame:
24     """FAIR Plan policies in force at 9/30 of the fiscal year, the only series that reaches
the 2024-25 events."""
25     df = con.execute(FAIR, [county, county]).df()
26     ca = df.groupby(["release", "y"]).fair.sum().reset_index().assign(scope="California")
27     return pd.concat([df[df.scope == county], ca], ignore_index=True)
30 def compute(con: duckdb.DuckDBPyConnection, county: str = "Contra Costa") -> pd.DataFrame:
31     df = con.execute(SQL, [county, county]).df()
32     ca = (
33         df.groupby(["release", "y"])["nonrenewed", "nonrenewed_insurer", "policies"]
34         .sum()
35         .reset_index()
36         .assign(scope="California")
37     )
38     df = pd.concat([df[df.scope == county], ca], ignore_index=True) # "California" rows above
ove exclude the county
39     rates = {"rate": df.nonrenewed / df.policies, "rate_insurer": df.nonrenewed_insurer / df.policies}
40     return pd.concat([df.assign(**rates), fair_series(con, county)], ignore_index=True).sort_values(["scope", "release", "y"])

```

- **5–12** Sum counts per release, year and scope (the county versus everyone else).
- **32–38** The SQL's "California" rows exclude the county; summing both groups gives the true statewide total, which replaces them.
- **39–40** Add the two rates and append the FAIR Plan series (tagged as its own release, `fair-plan-fy`).

`regulatory_alignment.py` is the event study:

```

9  FITS = (("2015-2021", "rate_insurer", "pts"), ("2020-2023", "rate", "pts"), ("fair-plan-f
y", "fair", "%"))
12 def distinct_events(ev: pd.DataFrame) -> pd.DataFrame:
13     ev = ev[ev.kind.isin(["fire", "regulation", "rate_decision"])]
14     ev["date"] = pd.to_datetime(ev.date)
15     g = ev.groupby(["kind", "date"], as_index=False).agg(
16         label=("dst_name", lambda s: ", ".join(sorted(set(s))[:4])), n=("dst", "nunique")
17     )
18     return g.assign(year=g.date.dt.year)
21 def study(ev: pd.DataFrame, tr: pd.DataFrame, release: str, measure: str, unit: str) -> pd.
DataFrame:
22     series = tr[tr.release == release].pivot(index="y", columns="scope", values=measure)
23     rows = []
24     for e in ev.itertuples():
25         for scope in series.columns:
26             b, d, a = (series[scope].get(e.year + k) for k in (-1, 0, 1))
27             if pd.isna(b) or pd.isna(d):
28                 continue # outside this release's window
29             rows.append({**e._asdict(), "scope": scope, "before": b, "during": d, "after":
a})
30     out = pd.DataFrame(rows, columns=[*ev.columns, "scope", "before", "during", "after"])
31     if unit == "%":
32         deltas = {"delta_during": (out.during / out.before - 1) * 100, "delta_after": (out.
after / out.before - 1) * 100}
33     else:
34         deltas = {"delta_during": (out.during - out.before) * 100, "delta_after": (out.afte
r - out.before) * 100}
35     return out.assign(**deltas, release=release, measure=measure, unit=unit)
38 def compute(con: duckdb.DuckDBPyConnection) -> pd.DataFrame:
39     ev, tr = distinct_events(events.compute(con)), trend.compute(con)
40     return pd.concat([study(ev, tr, *f) for f in FITS], ignore_index=True)

```

- **12-18** Collapse events to one per (kind, date): all fires declared the same day are one event, labelled by up to four names.
- **21-35** For each series (insurer-initiated rate, total rate, FAIR count) and each event: read the value in the year before, of, and after the event, for the county and the state; skip if the window is outside the series. Rates are reported as changes in percentage points, counts as percent change.

4.6 fair_mirror.py: finding 2

```

6  SQL = """
7  WITH nr AS (SELECT dst AS zip, year(valid_from) AS y, json_extract(props,'$.count')::INT AS
nonrenewed,
8              json_extract(props,'$.policies')::INT AS policies
9              FROM edges WHERE rel='NONRENEWED_IN' AND json_extract_string(props,'$.release')
=?),
10 fair AS (SELECT src AS zip, year(valid_from) AS y, json_extract(props,'$.count')::INT AS fa
ir
11           FROM edges WHERE rel='HAS_FAIR_PLAN_POLICIES')
12 SELECT substr(zip, 5) AS zip, json_extract_string(e.attrs,'$.county') AS county, y, nonrene
wed, policies, fair,
13        fair - lag(fair) OVER (PARTITION BY zip ORDER BY y) AS fair_delta,
14        fair::DOUBLE / (fair + policies) AS fair_share
15 FROM fair LEFT JOIN nr USING (zip, y) JOIN entities e ON e.entity_id = zip
16 """
19 def compute(con: duckdb.DuckDBPyConnection, release: str = "2020-2023") -> pd.DataFrame:
20     return con.execute(SQL, [release]).df().assign(release=release)

```

One row per ZIP and year with the FAIR Plan count, its year-over-year change (lag is a window function looking at the previous year of the same ZIP), and the FAIR share of all policies where voluntary counts exist. The left join keeps FY2024–2025 rows even though the CDI series has ended.

4.7 actor_centrality.py: finding 5

```
7 SQL = """
8 SELECT DISTINCT e.src, e.dst, e.rel FROM edges e
9 JOIN entities s ON s.entity_id = e.src JOIN entities t ON t.entity_id = e.dst
10 WHERE (s.type <> 'zip' OR json_extract_string(s.attrs,'$.county') = ?)
11 AND (t.type <> 'zip' OR json_extract_string(t.attrs,'$.county') = ?)
12 """
16 def compute(con: duckdb.DuckDBPyConnection, county: str = "Contra Costa") -> pd.DataFrame:
17     edges, names = con.execute(SQL, [county, county]).df(), con.execute(NAMES).df().set_index("entity_id")
18     g = nx.Graph()
19     g.add_edges_from(edges[["src", "dst"]].itertuples(index=False))
20     zips = {n for n in g if n.startswith("zip:")}
21     bc = nx.betweenness_centrality(g, normalized=True)
22     rows = [
23         {
24             "entity_id": n,
25             "type": names.type.get(n),
26             "name": names.canonical_name.get(n, n),
27             "degree": g.degree(n),
28             "betweenness": bc[n],
29             "cc_zips_within_2": sum(1 for z in zips if z != n and nx.has_path(g, n, z) and
30                                     nx.shortest_path_length(g, n, z) <= 2),
31             "rels": ", ".join(sorted(set(edges[(edges.src == n) | (edges.dst == n)].rel))),
32         }
33         for n in g
34         if n not in zips
35     ]
36     return pd.DataFrame(rows).sort_values(["betweenness", "degree"], ascending=False).reset_index(drop=True)
```

- **8–12** Take every edge whose endpoints are either non-ZIP entities or Contra Costa ZIPs. Dropping the other 1,600 ZIPs keeps the statewide hub from dominating for reasons that have nothing to do with the county.
- **18–21** Build an undirected NetworkX graph and compute normalised betweenness.
- **22–35** For every non-ZIP node: degree, betweenness, how many county ZIPs are within two hops, and which relations touch it. Sorted by betweenness.

4.8 evidence.py, explorer.py, geometry.py: serving tables

`evidence.py` joins every document edge to its document so the app can show the span and URL.
`explorer.py` builds the graph explorer's edge list:

```

6  SQL = """
7  WITH cc AS (SELECT entity_id FROM entities WHERE json_extract_string(attrs,'$.county') =
?),
8  det AS (
9    SELECT e.*, row_number() OVER (PARTITION BY src, rel, dst, extractor ORDER BY valid_from
DESC) AS rn
10   FROM edges e WHERE doc_id IS NULL AND (src IN (SELECT entity_id FROM cc) OR dst IN (SELEC
T entity_id FROM cc))
11  SELECT e.src, e.rel, e.dst, e.valid_from, e.valid_to, e.props, e.span, e.extractor, e.confidence, d.url,
12         s.type AS src_type, t.type AS dst_type, s.canonical_name AS src_name, t.canonical_name AS dst_name,
13         json_extract_string(s.attrs,'$.county') AS src_county, json_extract_string(t.attrs,'$.county') AS dst_county
14  FROM (SELECT * FROM edges WHERE doc_id IS NOT NULL UNION ALL SELECT * EXCLUDE (rn) FROM det
WHERE rn = 1) e
15  LEFT JOIN documents d USING (doc_id) JOIN entities s ON s.entity_id = e.src JOIN entities t
ON t.entity_id = e.dst
16  ORDER BY e.rel, e.src, e.dst
17  """

```

All document edges, plus the county ZIPs' deterministic edges collapsed to their latest year (the `row_number()` window numbers each ZIP's yearly edges from newest to oldest; `rn = 1` keeps the newest), with names, types and counties attached for drawing.

`geometry.py` writes the county's ZIP polygons for the map:

```

14 def county_geojson(con: duckdb.DuckDBPyConnection, zcta: gpd.GeoDataFrame, places: dict[str, str], county: str) -> dict:
15     zips = set(con.execute(ZIPS, [county]).df().zip)
16     g = zcta[zcta.zip.isin(zips)].copy()
17     g["geometry"] = g.geometry.simplify(TOLERANCE_M)
18     g["place"] = g.zip.map(places).fillna(g.zip)
19     return json.loads(g.to_crs(4326).to_json())

```

Select the county's ZIPs, simplify their outlines by 30 metres (halves the file), attach place names, and reproject to latitude/longitude, which web maps need.

4.9 The app: `policygraph/app.py`

```

6 ROOT = Path(__file__).resolve().parent.parent
7 sys.path.insert(0, str(ROOT)) # Streamlit Cloud puts only this file's directory on sys.pat
h
9 from policygraph.views import explorer, tables, timeline # noqa: E402
10 from policygraph.views import map as map_view # noqa: E402
11 from policygraph.views.data import DOCS # noqa: E402
12 st.set_page_config(page_title="policygraph", layout="wide")
13 st.title("Home—insurance non-renewals · Contra Costa County")
14 st.markdown("This is a knowledge graph built to answer one question: ...")
28 TABS = {
29     "Findings": lambda: st.markdown((DOCS / "FINDINGS.md").read_text()),
30     "Map": map_view.render,
31     "Timeline": timeline.render,
32     "Graph": explorer.render,
33     "Hazard residual": tables.hazard,
34     "Moratorium deferral": tables.moratorium,
35     "FAIR Plan mirror": tables.fair,
36     "Regulatory alignment": tables.regulatory,
37     "Actors": tables.actors,
38     "Evidence": tables.evidence,
39 }
40 for tab, render in zip(st.tabs(list(TABS)), TABS.values(), strict=True):
41     with tab:
42         render()

```

- **6–7** Streamlit Cloud runs the script with only its own folder on Python's import path. Inserting the repository root first makes `import policygraph.views` work everywhere. The `noqa: E402` comments tell the linter the late imports are deliberate.
- **13–15** Page setup, title, and the plain-language summary.
- **28–42** A dictionary from tab name to the function that draws it; `st.tabs` creates the tabs and each function draws inside its tab.

4.10 `views/data.py`: shared loading and colours

```

7 ROOT = Path(__file__).resolve().parent.parent
8 INSIGHTS, DOCS = ROOT / "data" / "insights", ROOT / "docs"
9 COUNTY = "Contra Costa"
12 SEQ = ["#cde2fb", "#9ec5f4", "#6da7ec", "#3987e5", "#256abf", "#184f95", "#0d366b"]
13 DIV = ["#0d366b", "#3987e5", "#9ec5f4", "#f0efec", "#f3a5a5", "#e34948", "#8f1f1f"]
14 CAT = {"blue": "#2a78d6", "orange": "#eb6834", "aqua": "#1baf7a", "yellow": "#eda100", "magenta": "#e87ba4", "violet": "#4a3aa7"}
15 KIND_COLOR = {"regulation": CAT["blue"], "fire": CAT["orange"], "moratorium": CAT["aqua"], "rate_decision": CAT["yellow"]}
30 @st.cache_data
31 def load(name: str) -> pd.DataFrame:
32     return pd.read_parquet(INSIGHTS / f"{name}.parquet")
44 def ramp(ramp_hex: list[str], t: float) -> list[int]:
45     """Piecewise-linear interpolation through the ramp at t in [0, 1]."""
46     t = min(max(t, 0.0), 1.0) * (len(ramp_hex) - 1)
47     i = min(int(t), len(ramp_hex) - 2)
48     a, b = hex_rgb(ramp_hex[i]), hex_rgb(ramp_hex[i + 1])
49     return [round(a[k] + (b[k] - a[k]) * (t - i)) for k in range(3)]

```

- **7–9** Paths derived from `__file__` again; the county name.
- **12–15** Colour scales: a single-hue blue ramp for magnitudes, a blue–grey–red ramp for values that can be positive or negative (residuals), and a fixed categorical order for series and event

kinds. These come from a validated colour-blind-safe reference palette.

- **30–32** `st.cache_data` keeps each parquet in memory after the first read so tabs render instantly.
- **44–49** Given a ramp and a position 0–1, blend between the two neighbouring colours.

4.11 `views/map.py`: the choropleth

```
13 def metrics() -> dict[str, tuple[pd.Series, bool, str]]:
14     res = load("hazard_residual")
15     res = res[(res.county == COUNTY) & (res.spec == "hazard")]
16     ins, tot = res[res.measure == "nonrenewed_insurer"].set_index("zip"), res[res.measure == "count"].set_index("zip")
17     fair = load("fair_mirror")
18     fair = fair[fair.county == COUNTY].set_index(["zip", "y"]).sort_index()
19     prot = load("explorer")
20     prot = prot[(prot.rel == "PROTECTED_BY") & (prot.src_county == COUNTY)]
21     covered = prot.groupby(prot.src.str[4:]).dst_name.agg(lambda s: "; ".join(sorted(set(s))))
22     return {
23         "Insurer-initiated non-renewal rate, 2021 (2015–2021 release)": (ins.rate * 100, False, "{:.2f} %"),
24         "Hazard residual, 2021 insurer-initiated (pts above the hazard line)": (ins.residual * 100, True, "{:+.2f} pts"),
25         ...
32         "Moratorium coverage (SB 824 bulletins)": (covered, False, "{}"),
33     }
```

Ten selectable metrics, each a Series indexed by ZIP, a flag saying whether it is diverging (needs the two-hue ramp), and a number format.

```
36 def colour(values: pd.Series, diverging: bool) -> dict[str, list[int]]:
37     if values.dtype == object:
38         return {z: [27, 175, 122, 190] for z in values.index} # categorical: covered or not
38     if diverging:
40         m = float(values.abs().max()) or 1.0
41         return {z: [*ramp(DIV, 0.5 + v / (2 * m)), 200] for z, v in values.items()}
42     lo, hi = float(values.min()), float(values.max())
43     return {z: [*ramp(SEQ, (v - lo) / ((hi - lo) or 1.0)), 200] for z, v in values.items() }
```

Text metrics (moratorium names) colour every covered ZIP green. Diverging metrics centre zero on the grey midpoint and scale symmetrically. Others map min→lightest, max→darkest. The fourth number is opacity.


```

46 def render() -> None:
47     label = st.selectbox("Colour ZIPs by", list(metrics()))
48     values, diverging, fmt = metrics()[label]
49     values = values.dropna()
50     fills = colour(values, diverging)
51     gj = copy.deepcopy(geojson())
52     for f in gj["features"]:
53         z = f["properties"]["zip"]
54         f["properties"]["fill"] = fills.get(z, [240, 239, 236, 120])
55         f["properties"]["display"] = fmt.format(values[z]) if z in values.index else "no data"
56         f["properties"]["label"] = label
57     layer = pdk.Layer("GeoJsonLayer", gj, pickable=True, stroked=True, filled=True,
                       get_fill_color="properties.fill", get_line_color=[82, 81, 78], line_width
th_min_pixels=1)
67     tooltip = {"html": "<b>{place}</b> · {zip}<br/>{label}<br/><b>{display}</b>"}
68     st.pydeck_chart(pdk.Deck(layers=[layer], initial_view_state=pdk.ViewState(**CENTER), to
oltip=tooltip, map_style="light"))

```

Pick a metric, compute fills, copy the cached GeoJSON (never mutate a cached object), stamp each polygon with its colour and formatted value, and draw a pydeck layer with hover tooltips. Below the map a legend line and a sortable table repeat the numbers, so colour never carries meaning alone.

4.12 views/timeline.py

```

13 def series_chart(tr: pd.DataFrame, ev: pd.DataFrame, release: str, measure: str, title: str) -> alt.Chart:
14     df = tr[tr.release == release].assign(pct=lambda d: d[measure] * 100, date=lambda d: pd.to_datetime(d.y.astype(str) + "-07-01"))
15     lo, hi = df.date.min() - pd.DateOffset(months=6), df.date.max() + pd.DateOffset(months=6)
16     ev = ev[(ev.date >= lo) & (ev.date <= hi)]
17     scale = alt.Scale(domain=[COUNTY, "California"], range=[CAT["blue"], CAT["orange"]])
18     line = alt.Chart(df).mark_line(point=alt.OverlayMarkDef(size=60), strokeWidth=2).encode(
19         (
20             x=alt.X("date:T", ...), y=alt.Y("pct:Q", title="% of policies"), color=alt.Color("scope:N", scale=scale, ...), tooltip=[...])
21         rules = alt.Chart(ev).mark_rule(strokeWidth=2, opacity=0.8).encode(x="date:T", color=alt.Color("kind:N", ...), tooltip=[...])
22     return (line + rules).properties(title=title, height=260).resolve_scale(color="independent")

```

Two Altair charts per release: the county and state rates as lines (annual points placed at mid-year), with the document events as vertical rules coloured by kind. `resolve_scale(color="independent")` lets the lines and rules use different colour legends. `render` draws one chart per release and a table of the events.

4.13 views/explorer.py

```

24 def neighbourhood(df: pd.DataFrame, start: str, hops: int) -> pd.DataFrame:
25     seen, frontier = {start}, {start}
26     for _ in range(hops):
27         step = df[df.src.isin(frontier) | df.dst.isin(frontier)]
28         frontier = (set(step.src) | set(step.dst)) - seen
29         seen |= frontier
30     return df[df.src.isin(seen) & df.dst.isin(seen)]

```

A breadth-first expansion: starting from one entity, take every edge touching the current frontier, add the new endpoints, repeat `hops` times, and return every edge among the reached nodes.

```

47 def render() -> None:
48     df = load("explorer")
49     ents = pd.concat([...]).drop_duplicates("id")
55     cc = set(df[df.src_county == COUNTY].src) | set(df[df.dst_county == COUNTY].dst)
56     pick = ents[(ents.type != "zip") | ents.id.isin(cc)].sort_values(["type", "name"])
57     default = int((pick.id == "zip:94549").to_numpy().argmax())
59     start = c1.selectbox("Start entity", pick.id, index=default, format_func=...)
60     hops = c2.slider("Hops", 1, 3, 1)
61     cap = c3.slider("Max edges", 50, 600, 200, step=50)
62     sub = neighbourhood(df, start, hops).head(cap)
65     nodes = [{"id": i, "label": ..., "title": i, "color": TYPE_COLOR.get(names.type[i], "#8
88"), "borderWidth": 3 if i == start else 1} for i in ids]
75     edges = [{"from": e.src, "to": e.dst, "label": e.rel, "title": title(e)} for e in sub.i
tertuples()]
80     st.iframe(HTML.format(vis=VIS, nodes=json.dumps(nodes), edges=json.dumps(edges)), heigh
t=640)

```

The picker offers every non-ZIP entity plus the county's ZIPs (Lafayette by default). The chosen neighbourhood is turned into node and edge lists for vis-network, a JavaScript graph library loaded from a CDN inside an iframe. `title(e)` builds the hover text: relation, extractor, confidence, dates, props, the quoted span and the source URL, the receipt for every edge.

4.14 views/tables.py

Five small functions, one per table tab. `hazard` lets you choose the measure and the regressor spec, prints the fit summary (coefficients with p-values, R^2 , sample size), shows the county's ZIPs with their rank among all California ZIPs, and draws a scatter of rate against Very High share with the county highlighted. `moratorium` picks a release and shows protected-versus-control summaries with ratios, then the per-ZIP rows. `fair`, `regulatory`, `actors` and `evidence` show their tables with a caption explaining what they are.

Part 5. The extraction service

The service is the only code that talks to a language model. It is a separate program with two processes:

- **the API** (`api.py`, run by `uvicorn`): accepts jobs, reports run manifests, serves artifacts, and answers resolution requests;
- **the worker** (`worker.py`): takes queued jobs one at a time and produces a validated artifact per document.

They share a SQLite queue and a folder of JSON artifacts (`store.py`). The design idea is that the model is boxed in on both sides: deterministic *anchors* go in (every ZIP, date and bill number the text contains, plus every fire heading with a ZIP list under it), and deterministic *validators* check what comes out (spans must be verbatim, types must fit the relation, ZIPs and dates must be anchored). What survives is written as an immutable artifact whose id is a hash of everything that produced it.

5.1 `extraction_service/__init__.py` and `models.py`

```
4 STORE = Path(os.environ.get("EXTRACTION_STORE", "data/extractions"))
5 SCHEMA_DIR = Path(__file__).parent / "schema"
6 PROMPT_DIR = Path(__file__).parent / "prompts"
```

Where artifacts go, and where the JSON schema and the prompt live.

```
6 EntityType = Literal["ZIP", "Insurer", "Fire", "Moratorium", "Regulation", "Official", "RateFiling", "County"]
7 Rel = Literal["PROTECTED_BY", "TRIGGERED", "ISSUED", "FILED", "DECIDED", "AUTHORED", "DONATED_TO"]
9 REL_TYPES: dict[str, tuple[set[str], set[str]]] = {
10     "PROTECTED_BY": ({ "ZIP" }, { "Moratorium" }),
11     "TRIGGERED": ({ "Fire" }, { "Moratorium" }),
12     "ISSUED": ({ "Official" }, { "Moratorium", "Regulation" }),
13     "FILED": ({ "Insurer" }, { "RateFiling" }),
14     "DECIDED": ({ "Official" }, { "RateFiling" }),
15     "AUTHORED": ({ "Official" }, { "Regulation" }),
16     "DONATED_TO": ({ "Insurer" }, { "Official" }),
17 }
```

The closed vocabularies: eight entity types, seven relations, and for each relation which source and target types are allowed. `Literal[...]` makes Pydantic reject any other value.

```

20 class Entity(BaseModel):
21     mention: str
22     type: EntityType
25 class Edge(BaseModel):
26     src_mention: str
27     rel: Rel
28     dst_mention: str
29     valid_from: date | None = None
30     valid_to: date | None = None
31     props: dict = Field(default_factory=dict)
32     span: str = Field(min_length=8)
33     confidence: float = Field(ge=0, le=1)
36 class Extraction(BaseModel):
37     entities: list[Entity]
38     edges: list[Edge]
41 class Artifact(Extraction):
42     extraction_id: str
43     doc_id: str
44     schema_v: str
45     prompt_v: str
46     model_id: str
47     status: Literal["ok", "no_target_relations", "failed_validation", "error"]
48     rejected: list[str] = Field(default_factory=list)
51 class Job(BaseModel):
52     run_id: str
53     doc_id: str
54     source: str
55     text: str
56     force: bool = False # bypass the extraction cache and overwrite the artifact

```

- **20–38** What the model returns: entities (a mention and its type) and edges (two mentions, a relation, optional dates, a property bag, a span of at least eight characters, and a confidence in 0–1).
- **41–48** An artifact is an extraction plus its identity (which document, schema version, prompt version, model), a status, and the list of edges the validators rejected (kept for audit).
- **51–56** A job as the API receives it.

```

59 class ZipBlock(BaseModel):
60     heading: str
61     name: str
62     date: date | None # the declaration this block sits under; None outside a dated section
63     zips: list[str]
64     span: str
67 class Anchors(BaseModel):
68     zips: set[str]
69     dates: set[date]
70     bills: set[str]
71     zip_blocks: list[ZipBlock] = Field(default_factory=list)
72     def for_prompt(self) -> str:
73         """Headings and dates only: the model relates fires to declarations, it does not enumerate ZIPs."""
74         blocks = [{"heading": b.heading, "declaration": str(b.date), "zips": len(b.zips)} for b in self.zip_blocks]
75         return self.model_dump_json(exclude={"zip_blocks"})[:-1] + f', "fire_blocks": {blocks}}'.replace('"', '')

```

- **59–64** A ZIP block: a fire heading in a bulletin, the fire's cleaned name, the declaration date the section falls under, the ZIPs listed under it, and the verbatim text of heading plus list.

- **67–76** The anchors handed to the model. `for_prompt` serialises them as JSON but replaces each block's ZIP list with a count: the model is told which fires exist and which declaration they belong to, not asked to copy hundreds of ZIPs.

```

79 class Candidate(BaseModel):    # an entity the resolver may map a mention to
86 class Mention(BaseModel):     # a mention plus a context span
92 class Resolution(BaseModel):  # the service's answer: entity id or None, method, confidence, nearest candidate and its cosine, models used
104 class ResolveRequest(BaseModel): mentions + candidates

```

The request and response shapes of the resolve endpoint.

5.2 `chunk.py`

```

3 TARGET, OVERLAP = 5000, 1500 # ~10 fires per chunk: the model drops ZIPs past ~200 per call; overlap covers a whole ZIP list
4 HEAD = 3500 # the document head travels with every chunk as context: bulletins date their declarations on page 2-3
14 def split(text: str, target: int = TARGET, overlap: int = OVERLAP) -> list[Chunk]:
15     if len(text) <= target:
16         return [Chunk(0, 0, text)]
17     chunks: list[Chunk] = []
18     start = 0
19     while start < len(text):
20         end = min(len(text), start + target)
21         if end < len(text) and (cut := text.rfind("\n", start + target // 2, end)) > 0:
22             end = cut
23         chunks.append(Chunk(len(chunks), start, text[start:end]))
24         if end == len(text):
25             break
26         start = end - overlap
27     return chunks

```

Long documents are cut into overlapping pieces. Each piece is at most 5,000 characters, preferably ending at a line break in its second half, and the next piece starts 1,500 characters back so that any ZIP list cut in two appears whole in one of them. Each chunk remembers its starting offset so spans can be located in the document. Separately, the first 3,500 characters of the document ride along with every chunk as read-only context, because bulletins state their declaration dates near the top and list ZIPs pages later.

5.3 `anchors.py`

```

8 ZIP_RE = re.compile(r"\b9[0-6]\d{3}\b")
9 BILL_RE = re.compile(r"\b(SB|AB|Senate Bill|Assembly Bill)\s?(\d{1,4})\b", re.I)
10 HOUSE = {"senate bill": "SB", "assembly bill": "AB"}
11 DATE_RE = re.compile(r"\b[A-Z][a-z]+\s\d{1,2},\s\d{4}\b|\b\d{4}-\d{2}-\d{2}\b")
13 DECL_RE = re.compile(r"Governor['']\s+(\b[A-Z][a-z]+\s\d{1,2},\s\d{4})\s+(?:emergency\s+)?[Dd]\s?eclarations?")
14 HEADING_RE = re.compile(r"^[^\n]{1,90}\bFires?\b[^\n]{0,60}$")
15 NOT_HEADING = re.compile(r"perimeter|Protection|Department|insurer|ZIP|Governor|declar|Bulletin|Moratorium|Page|due to|•", re.I)
16 NAME_STRIP = (re.compile(r"\(.*?\)"), re.compile(r"[\[\]]"), re.compile(r"\d+$"), re.compile(r"\bFires?\b(\s+\d{4})?\s*$"))

```

- **8** A California ZIP: five digits starting 90–96, as a whole word.
- **9–10** Bill numbers in short or long form, normalised to SB824.
- **11** Dates written as January 7, 2025 or 2025-01-07.
- **13** The sentence that dates a bulletin section: "due to the Governor's October 27, 2019 Declaration" or "as a result of the Governor's September 6, 2020 emergency declaration" (pypdf sometimes inserts a space inside "declaration", hence `[Dd]\s?eclaration`).
- **14–15** A fire heading is a short line containing "Fire" or "Fires" that is not one of the sentences that merely mention fires.
- **16** Patterns stripped from a heading to get the fire's name: parentheticals, square brackets, trailing footnote digits, and the word "Fire(s)" with an optional year.

```

29 def fire_name(heading: str) -> str:
30     """[Old] Water Fire' -> 'Old Water'; 'SHF Lightning Fires 2020' -> 'SHF Lightning'."""
31     name = heading.strip()
32     for rx in NAME_STRIP:
33         name = rx.sub("", name).strip()
34     return " ".join(name.split())
37 def zip_blocks(text: str) -> list[ZipBlock]:
38     """Fire headings followed by ZIP lines, dated by the nearest preceding declaration sent
ence. Bulletins only."""
39     decls = [(m.start(), pd.to_datetime(m.group(1)).date()) for m in DECL_RE.finditer(text)]
40     lines, pos, out = text.split("\n"), 0, []
41     starts = []
42     for line in lines:
43         starts.append(pos)
44         pos += len(line) + 1
45     i = 0
46     while i < len(lines):
47         line = lines[i].strip()
48         is_heading = HEADING_RE.match(line) and not NOT_HEADING.search(line) and not ZIP_RE.search(line)
49         if is_heading and i + 1 < len(lines) and ZIP_RE.search(lines[i + 1]):
50             j = i + 1
51             while j < len(lines) and ZIP_RE.search(lines[j]):
52                 j += 1
53             zips = [z for k in range(i + 1, j) for z in ZIP_RE.findall(lines[k])]
54             before = [d for p, d in decls if p < starts[i]]
55             span = text[starts[i] : starts[j - 1] + len(lines[j - 1])].strip()
56             out.append(ZipBlock(heading=line, name=fire_name(line), date=before[-1] if before else None, zips=zips, span=span))
57             i = j
58         else:
59             i += 1
60     return out

```

- **39** Every declaration sentence with its character position and date.
- **40–44** Split into lines and record where each line starts in the full text.
- **46–60** Scan the lines. A heading immediately followed by a line containing a ZIP starts a block; the block continues while lines contain ZIPs. Collect the ZIPs, find the last declaration sentence before the heading (that is the section's date), cut the exact text from heading to last ZIP line as the span, and jump past the block.

On the four bulletins this parser recovers every ZIP list: 231 ZIPs in the 2019 bulletin and 713 in the 2020-11 bulletin, matching the hand-made golden set exactly. The 2025-1 bulletin lists ZIPs on the

same line as the fire name, so it yields no blocks and is handled entirely by the model.

```
63 def extract(text: str) -> Anchors:
64     return Anchors(zips=set(ZIP_RE.findall(text)), dates=_dates(text),
                     bills={HOUSE.get(h.lower(), h.upper()) + n for h, n in BILL_RE.findall(text)}, zip_blocks=zip_blocks(text))
```

All anchors for a piece of text.

5.4 validators.py

```
11 def _norm(s: str) -> str:
12     """pypdf breaks words across lines ("Sanda\nlwood"), so whitespace is ignored entirely;
   characters are not."""
13     return _WS.sub("", s).lower()
17 def span_verbatim(edge: Edge, chunk: str) -> str | None:
18     return None if _norm(edge.span) in _norm(chunk) else "span is not a verbatim substring
   of the chunk"
21 def types_allowed(edge: Edge, types: dict[str, str]) -> str | None:
22     src_ok, dst_ok = REL_TYPES[edge.rel]
23     got = (types.get(edge.src_mention), types.get(edge.dst_mention))
24     if got[0] in src_ok and got[1] in dst_ok:
25         return None
26     return f"types {got[0]}->{got[1]} not allowed for {edge.rel}; need {'/'.join(src_ok)}->
   {'/'.join(dst_ok)}"
```

Each check returns `None` on success or a message on failure; the message is fed back to the model on retry.

- **11–18** The span must be a verbatim substring of the chunk, comparing with all whitespace removed and case ignored (PDF text extraction breaks words across lines; nothing else is forgiven).
- **21–26** The source and target mention types must be allowed for the relation.

```
29 def _near(d: date, anchors: Anchors) -> bool:
30     return d in anchors.dates or any(abs(d - a) <= timedelta(days=366) for a in anchors.dates)
33 def anchored(edge: Edge, anchors: Anchors) -> str | None:
34     for m in (edge.src_mention, edge.dst_mention):
35         if _ZIP.fullmatch(m) and m not in anchors.zips:
36             return f"anchors: ZIP {m} is not in the chunk"
37         if _BILL.fullmatch(m) and m.upper().replace(" ", "") not in anchors.bills:
38             return f"anchors: bill {m} is not in the chunk"
39     if all(d is None or _near(d, anchors) for d in (edge.valid_from, edge.valid_to)):
40         return None
41     return "anchors: dates are not within a year of any date in the chunk"
44 def dates_sane(edge: Edge) -> str | None:
45     if edge.valid_from is None or edge.valid_to is None or edge.valid_from <= edge.valid_to:
46         return None
47     return "dates: valid_from > valid_to"
```

- **33–41** A ZIP or bill mention must appear in the chunk (this is what stops a hallucinated ZIP), and any edge date must be within a year of some date the chunk (or document head)

contains.

- **44–47** Start must not be after end.

```
50 def validate(x: Extraction, chunk: str, anchors: Anchors) -> tuple[Extraction, list[str]]:
51     types = {e.mention: e.type for e in x.entities}
52     kept, errors = [], []
53     for e in x.edges:
54         failed = [m for m in (span_verbatim(e, chunk), types_allowed(e, types), anchored(e,
55 anchors), dates_sane(e)) if m]
56         if failed:
57             errors.append(f"{e.src_mention} -{e.rel}-> {e.dst_mention}: {'; '.join(failed)}")
58         else:
59             kept.append(e)
60     return Extraction(entities=x.entities, edges=kept), errors
```

Run all four checks on every edge; keep the clean ones, collect a readable error for each rejected one.

5.5 `expand.py`: deterministic expansion of ZIP blocks

```
8 FIRE_RE = re.compile(r"^(.+?) Fires(?: \(..*?\))?(?: \{d{4}\})?(?: \{d{4}\})?$", re.I)
9 MOR_RE = re.compile(r"^(.+?) Fires(?: \(..*?\))?(?: \{d{4}\})? moratorium (\{d{4}\}-\{d{2}\}-\{d{2}\})$", re.I)
10 CONFIDENCE = 0.97 # the ZIP list is read by regex; the fire-to-declaration link is the model's or the section's
13 def same(a: str, b: str) -> bool:
14     a, b = a.lower(), b.lower()
15     return a == b or a.endswith(b) or b.endswith(a)
```

- **8–9** Tolerant versions of the mention conventions: they accept parentheticals, "Fires", and a doubled year, so that variants the model copies from headings can be recognised.
- **13–15** Two names are "the same" if equal or one is a suffix of the other. This absorbs PDF artefacts such as "Iwood" for "Sandalwood" and "Water" for "Old Water".

```
18 def canonical_mentions(x: Extraction) -> dict[str, str]:
22     parsed: dict[str, tuple[str, str, str | None]] = {}
23     for e in x.entities:
24         if e.type == "Fire" and (m := FIRE_RE.match(e.mention)):
25             parsed[e.mention] = ("Fire", fire_name(m.group(1) + " Fire"), m.group(2))
26         elif e.type == "Moratorium" and (m := MOR_RE.match(e.mention)):
27             parsed[e.mention] = ("Moratorium", fire_name(m.group(1) + " Fire"), m.group(2))
28     out = {}
29     for mention, (kind, name, when) in parsed.items():
30         peers = [(n, w) for k, n, w in parsed.values() if k == kind and same(n, name) and
31 (w == when or when is None or w is None)]
32         name = max((n for n, _ in peers), key=len)
33         when = when or next((w for _, w in peers if w), None)
34         if kind == "Fire":
35             out[mention] = f"{name} Fire {when}" if when else f"{name} Fire"
36         else:
37             out[mention] = f"{name} Fire moratorium {when}"
38     return {k: v for k, v in out.items() if v != k}
```


Builds a rename map from every fire or moratorium mention the model produced to its canonical form: parse each into (kind, clean name, year or date); group with its peers (same kind, same-ish name, compatible date); take the longest name in the group and any date the group knows; rebuild the mention in the prompt's convention. Only mentions that actually change are returned. Examples: "Eagle Fire (Lake County) 2019" → "Eagle Fire 2019"; "Bobcat Fire" → "Bobcat Fire 2020"; "Iwood Fire moratorium 2019-10-11" → "Sandalwood Fire moratorium 2019-10-11".

```

42 def expand(x: Extraction, blocks: list[ZipBlock]) -> Extraction:
43     fold = canonical_mentions(x)
44     ents = {(fold.get(e.mention, e.mention), e.type): Entity(mention=fold.get(e.mention, e.mention), type=e.type) for e in x.entities}
45     edges: dict[tuple, Edge] = {}
46     for e in x.edges:
47         e = e.model_copy(update={"src_mention": fold.get(e.src_mention, e.src_mention), "dst_mention": fold.get(e.dst_mention, e.dst_mention)})
48         edges.setdefault((e.src_mention, e.rel, e.dst_mention), e)
49     mors = {m: MOR_RE.match(m) for (m, t) in ents if t == "Moratorium" and MOR_RE.match(m)}
50     official = next((e.src_mention for e in edges.values() if e.rel == "ISSUED" and e.dst_mention in mors), None)
51     for b in blocks:
52         if b.date is None:
53             continue
54         hit = [m for m, r in mors.items() if same(r.group(1), b.name) and r.group(2) == b.date.isoformat()]
55         name = mors[hit[0]].group(1) if hit else b.name
56         mor, fire = hit[0] if hit else f"{name} Fire moratorium {b.date.isoformat()}", f"{name} Fire {b.date.year}"
57         base = {"valid_from": b.date, "valid_to": _plus_year(b.date), "props": {"expanded_from": "zip_block", "span": b.span, "confidence": CONFIDENCE}}
58         ents.setdefault((fire, "Fire"), Entity(mention=fire, type="Fire"))
59         ents.setdefault((mor, "Moratorium"), Entity(mention=mor, type="Moratorium"))
60         edges.setdefault((fire, "TRIGGERED", mor), Edge(src_mention=fire, rel="TRIGGERED", dst_mention=mor, **base))
61         if official:
62             edges.setdefault((official, "ISSUED", mor), Edge(src_mention=official, rel="ISSUED", dst_mention=mor, **base))
63         for z in b.zips:
64             ents.setdefault((z, "ZIP"), Entity(mention=z, type="ZIP"))
65             edges.setdefault((z, "PROTECTED_BY", mor), Edge(src_mention=z, rel="PROTECTED_BY", dst_mention=mor, **base))
66     return Extraction(entities=list(ents.values()), edges=list(edges.values()))

```

- **45–54** Apply the rename map to entities and edges, and de-duplicate edges by (source, relation, target), the model's own edge wins because it is inserted first (`setdefault` keeps the first value).
- **55–56** Index the moratorium mentions, and find the official who issued them (the Commissioner).
- **57–77** For every dated block: find the model's moratorium mention for this fire (same name, same declaration date) or synthesise one from the heading; then add, only where missing, the TRIGGERED edge, the ISSUED edge and one PROTECTED_BY edge per ZIP. Expanded edges carry the block's verbatim span, a confidence of 0.97, the moratorium's one-year window, and `props.expanded_from = "zip_block"` so downstream code can tell them apart.

This step is why recall on bulletins is 0.997: the model reliably names the fires and dates; the regex reliably reads the lists; neither is asked to do the other's job.

5.6 The adapters: `adapters/__init__.py`, `openai_compat.py`, `anthropic.py`

```
7 class Adapter(Protocol):
8     model_id: str
10     def structured(self, prompt: str, schema: dict) -> dict:
11         """One model call constrained to a JSON schema; the caller validates the result."""
15 def build(model_id: str) -> Adapter:
16     if PROVIDER == "anthropic": ... return AnthropicAdapter(model_id)
20     if PROVIDER == "openai_compat": ... return OpenAICompatAdapter(model_id)
24     raise ValueError(f"unknown EXTRACTION_PROVIDER {PROVIDER!r}")
```

An adapter is anything with a `model_id` and a `structured(prompt, schema)` method returning a dictionary. `build` picks the implementation from the `EXTRACTION_PROVIDER` variable. Everything above this line is provider-agnostic.

```
10 def client(transport: httpx.BaseTransport | None = None) -> httpx.Client:
11     return httpx.Client(base_url=os.environ["OPENAI_COMPAT_BASE_URL"],
                          headers={"Authorization": f"Bearer {os.environ['OPENAI_COMPAT_API_KEY']}"},
                          timeout=httpx.Timeout(600, read=120), transport=transport)
27 def structured(self, prompt: str, schema: dict) -> dict:
28     for attempt in range(RETRIES):
29         try:
30             return self._once(prompt, schema)
31         except httpx.TransportError as e:
32             if attempt == RETRIES - 1:
33                 raise
34             print(f"retry {attempt + 1}/{RETRIES} after {type(e).__name__}", flush=True)
35             time.sleep(BACKOFF_S * (attempt + 1))
38     def _once(self, prompt: str, schema: dict) -> dict:
39         body = {"model": self.model_id, "max_tokens": self.max_tokens, "temperature": 0, "stream": True,
                "messages": [{"role": "user", "content": prompt}],
                "response_format": {"type": "json_schema", "json_schema": {"name": "result", "schema": schema}}}
47         parts: list[str] = []
48         with self.client.stream("POST", "/chat/completions", json=body) as r:
49             r.raise_for_status()
50             for line in r.iter_lines():
51                 if line.startswith("data: ") and line != "data: [DONE]":
52                     choices = json.loads(line[6:])["choices"] # the final usage event carries no choices
53                     parts += [c["delta"].get("content") or "" for c in choices]
54         return json.loads("".join(parts))
```

- **10–16** An HTTP client for any OpenAI-compatible endpoint (Fireworks here); the `transport` parameter lets tests substitute a fake server. The read timeout is the maximum silence between streamed tokens.
- **27–36** Up to three attempts; a network stall waits 15, then 30 seconds before retrying.
- **38–54** One call: temperature 0, streaming on, output constrained to the JSON schema. The stream arrives as `data: {...}` lines; the text deltas are concatenated and parsed as JSON.

The Anthropic adapter does the same through Anthropic's SDK, using a forced tool call whose input schema is the JSON schema.

5.7 `embed.py` and `resolve.py`: the resolver's model steps

```
15     def embed(self, texts: list[str]) -> list[list[float]]:
16         r = self.client.post("/embeddings", json={"model": self.model_id, "input": texts})
17         r.raise_for_status()
18         return [d["embedding"] for d in sorted(r.json()["data"], key=lambda d: d["index"])]
```

Embeds a list of strings in one request and returns the vectors in input order.

```
10 ACCEPT, MARGIN = 0.92, 0.05 # cosine to accept on embeddings alone, and the gap to the run
   ner-up
11 JUDGE_BAND, JUDGE_ACCEPT = 0.75, 0.85 # below the band nothing is asked; the judge must be
   this sure
19 JUDGE_PROMPT = """Do the mention and the candidate refer to the same real-world {type}?
20 Mention: {mention}
21 Mention context: {context}
22 Candidate: {name} – {description}
23 Answer with same_entity and your confidence."""
28 def cosines(q: np.ndarray, m: np.ndarray) -> np.ndarray:
29     return (m @ q) / (np.linalg.norm(m, axis=1) * np.linalg.norm(q))
44 def decide(m: Mention, order: list[tuple[Candidate, float]], judge: Judge, models: tuple[str],
   r, str) -> Resolution:
46     if not order:
47         return Resolution(**base, entity_id=None, method="unresolved", confidence=0.0)
48     (top, s1), s2 = order[0], order[1][1] if len(order) > 1 else 0.0
49     if s1 >= ACCEPT and s1 - s2 >= MARGIN:
50         return Resolution(**base, entity_id=top.entity_id, method="embedding", confidence=s
   1, nearest=top.entity_id, score=s1)
51     for c, s in order[:2]:
52         if s >= JUDGE_BAND and (p := judge_one(m, c, judge)) >= JUDGE_ACCEPT:
53             return Resolution(**base, entity_id=c.entity_id, method="llm_judge", confidence
   =p, nearest=top.entity_id, score=s1)
54     return Resolution(**base, entity_id=None, method="unresolved", confidence=0.0, nearest=
   top.entity_id, score=s1)
```

The decision rule for one mention, given candidates ranked by cosine similarity:

- Accept on embeddings alone if the best cosine is at least 0.92 *and* beats the runner-up by 0.05.
- Otherwise, for the top two candidates with cosine at least 0.75, ask the model "same entity?" with the mention's context; accept if it says yes with confidence at least 0.85.
- Otherwise unresolved, recording the nearest candidate and its score so a reviewer can see what was close.

```

62 def resolve(mentions, candids, embed, judge, models) -> list[Resolution]:
66     cache = STORE / "resolutions"
68     keys = [cache_key(m, [c for c in candids if c.type == m.type], models) for m in mentions]
69     todo = [m for m, k in zip(mentions, keys, strict=True) if not (cache / f"{k}.json").exists()]
70     if todo:
71         vecs = np.array(embed([m.mention for m in todo] + [f"{c.name} - {c.description}" for c in candids]))
72         mv, cv = vecs[: len(todo)], vecs[len(todo) :]
73         for m, v in zip(todo, mv, strict=True):
74             same = [i for i, c in enumerate(candids) if c.type == m.type]
75             order = ranked(v, [candids[i] for i in same], cv[same]) if same else []
76             r = decide(m, order, judge, models)
77             (cache / f"{cache_key(m, [candids[i] for i in same], models)}.json").write_text(
                r.model_dump_json(indent=1))
78     return [Resolution.model_validate_json((cache / f"{k}.json").read_text()) for k in keys]

```

Every answer is cached by a hash of the mention, its context, the same-type candidates, the two model ids and a resolver version, so identical questions never cost a second call. Mentions and candidates are embedded in one batch; each mention is compared only with candidates of its own type (an insurer can never resolve to an official).

5.8 store.py: queue and artifact store

```

6 DDL = """
7 CREATE TABLE IF NOT EXISTS runs (run_id TEXT PRIMARY KEY, schema_v TEXT, created_at TEXT DEFAULT current_timestamp);
8 CREATE TABLE IF NOT EXISTS jobs (run_id TEXT, doc_id TEXT, source TEXT, text TEXT, status TEXT,
9     extraction_id TEXT, force INTEGER DEFAULT 0, PRIMARY KEY (run_id, doc_id));
10 """
39 def claim() -> tuple[str, str, str, str, bool] | None:
40     con = _con()
41     con.execute("BEGIN IMMEDIATE")
42     row = con.execute("SELECT run_id, doc_id, source, text, force FROM jobs WHERE status='queued' LIMIT 1").fetchone()
43     if row:
44         con.execute("UPDATE jobs SET status='running' WHERE run_id=? AND doc_id=?", row[:2])
45     con.execute("COMMIT")
46     return (*row[:4], bool(row[4])) if row else None
49 def complete(run_id: str, doc_id: str, art: Artifact, overwrite: bool = False) -> None:
50     path = STORE / f"{art.extraction_id}.json"
51     if overwrite or not path.exists():
52         path.write_text(art.model_dump_json(indent=1))
53     _con().execute("UPDATE jobs SET status=?, extraction_id=? WHERE run_id=? AND doc_id=?",
        (art.status, art.extraction_id, run_id, doc_id))
59 def manifest(run_id: str) -> dict:
    ... returns run_id, schema_v, status ("running" while any job is queued or running, else "complete"),
    and the extraction ids grouped as extractions / failed / errors

```

A SQLite file holds runs and jobs. `claim` takes one queued job inside an immediate transaction so two workers could not grab the same one. `complete` writes the artifact (never overwriting unless forced) and records its id on the job. `requeue_running` (called at worker start) returns jobs a crashed worker left behind to the queue; `fail` marks a job `error` with the message.

5.9 worker.py

```
14 MODEL_ID = os.environ.get("EXTRACTION_MODEL", "accounts/fireworks/models/gpt-oss-120b")
15 SCHEMA_V, PIPELINE_V = "1.0", "0.3.2"
16 PROMPT = (PROMPT_DIR / "v2.md").read_text()
17 PROMPT_V = hashlib.sha256(PROMPT.encode()).hexdigest()[:12]
18 SCHEMA = json.loads((SCHEMA_DIR / "v1.json").read_text())
21 def extraction_id(doc_id: str, model_id: str) -> str:
22     return hashlib.sha256("|".join((doc_id, SCHEMA_V, PROMPT_V, model_id, "adaptive", PIPELINE_V)).encode()).hexdigest()
```

- **15–17** The schema version, the pipeline version (bumped whenever the worker's logic changes), and the prompt version, which is a hash of the prompt text so any edit changes it.
- **21–22** An artifact's id is a hash of everything that determines its content: document, schema, prompt, model, decoding mode and pipeline version. Same inputs → same id → cache hit; any change → a new id, and the old artifact is never touched.

```
25 def merge(parts: list[Extraction]) -> Extraction:
26     ents = {(e.mention, e.type): e for p in parts for e in p.entities}
27     edges: dict[tuple, Edge] = {}
28     for e in (e for p in parts for e in p.edges):
29         k = (e.src_mention, e.rel, e.dst_mention) # one edge per triple within a document
30         if k not in edges or e.confidence > edges[k].confidence:
31             edges[k] = e
32     return Extraction(entities=list(ents.values()), edges=list(edges.values()))
```

Combine the chunks' results: unique entities; for edges that overlapping chunks both produced, keep the more confident one.

```
35 def process(doc_id: str, text: str, adapter: Adapter, force: bool = False) -> Artifact:
36     xid = extraction_id(doc_id, adapter.model_id)
37     if store.exists(xid) and not force:
38         return store.load(xid)
39     parts, errors = [], []
40     head = text[:HEAD]
41     for ch in split(text):
42         anc = anchors_for(ch.text)
43         anc.dates |= anchors_for(head).dates # declaration dates live in the head; ZIPs and bills must be in the chunk
44         prompt = PROMPT.format(anchors=anc.for_prompt(), head=head, chunk=ch.text)
45         ok, errs = validate(Extraction.model_validate(adapter.structured(prompt, SCHEMA)), ch.text, anc)
46         parts.append(ok)
47         if errs: # keep the first pass's valid edges; the retry only needs to recover the rejected ones
48             retry = prompt + "\n\nThese edges were rejected; re-emit only corrected versions of them:\n" + "\n".join(errs)
49             ok, errs = validate(Extraction.model_validate(adapter.structured(retry, SCHEMA)), ch.text, anc)
50             parts.append(ok)
51         errors += errs
52     merged = expand(merge(parts), anchors_for(text).zip_blocks)
53     status = "ok" if merged.edges else "failed_validation" if errors else "no_target_relations"
54     return Artifact(**merged.model_dump(), extraction_id=xid, doc_id=doc_id, schema_v=SCHEMA_V, prompt_v=PROMPT_V, model_id=adapter.model_id, status=status, rejected=errors)
```

The per-document pipeline:

- **36–38** Cache check by extraction id.
- **40–44** For each chunk: compute anchors (ZIPs and bills from the chunk, dates also from the document head), fill the prompt template.
- **45–51** Call the model, validate; if anything was rejected, call once more with the rejection reasons appended, asking only for corrected versions; keep everything valid from both passes; remember the final rejections.
- **52** Merge the chunks and run the ZIP-block expansion over the whole document.
- **53–54** Status and artifact.

```
66 def main() -> None:
67     adapter = build(MODEL_ID)
68     store.requeue_running()
69     while True:
70         if (job := store.claim()) is None:
71             time.sleep(1)
72             continue
73         run_id, doc_id, _, text, force = job
74         try:
75             store.complete(run_id, doc_id, process(doc_id, text, adapter, force), overwrite=force)
76         except Exception as e: # a bad document must not stop the queue; the manifest reports it
77             store.fail(run_id, doc_id, f"{type(e).__name__}: {e}")
78             print(f"error {doc_id[:12]}: {type(e).__name__}: {e}", flush=True)
```

The worker loop: requeue anything a previous worker abandoned, then forever claim a job, process it, store the result; any exception marks that job as an error and the loop continues.

5.10 `api.py`

Five endpoints on a FastAPI app: `POST /v1/runs` creates a run id; `POST /v1/jobs` enqueues a job; `GET /v1/manifests/{run_id}` reports progress and the artifact ids; `GET /v1/extractions/{id}` returns an artifact; `POST /v1/resolve` runs the embedding and judge steps synchronously for a batch of mentions and returns the resolutions. FastAPI validates every request body against the Pydantic models.

5.11 The prompt (`prompts/v2.md`) and schema (`schema/v1.json`)

The prompt states the rules the validators enforce (verbatim spans, anchored ZIPs and dates, closed relation set, one edge per triple), the special rules for bulletins (one TRIGGERED, one ISSUED and one PROTECTED_BY per ZIP for each fire; PROTECTED_BY points at the moratorium, never the fire), for press releases (the Commissioner ISSUED what the release announces; a request is FILED; a ruling or settlement is DECIDED with an outcome) and the mention conventions the resolver keys on. Then it includes the document head, the anchors and the chunk. Everything in braces is filled by `PROMPT.format(...)`; literal braces in the examples are doubled (`{{ }}`) so `format` leaves them alone.

The schema is the JSON shape the API forces the model to produce: an `entities` array and an `edges` array with exactly the fields the Pydantic models expect, enumerated types and relations, and `additionalProperties: false` so nothing extra sneaks in.

5.12 `eval/__init__.py`: scoring against the golden set

```
12 def keys(x: Extraction) -> set[tuple]:
13     return {(e.src_mention.strip().lower(), e.rel, e.dst_mention.strip().lower()) for e in
x.edges}
20 def main() -> None:
21     adapter = build(MODEL_ID)
22     total, by_rel = Counter(), {}
23     for case in sorted(GOLDEN.glob("*.json")):
24         g = json.loads(case.read_text())
25         art = process(g["doc_id"], g["text"], adapter)
26         pred, gold = keys(art), keys(Extraction.model_validate(g["expected"]))
27         c = Counter(tp=len(pred & gold), fp=len(pred - gold), fn=len(gold - pred))
```

Each golden file holds a document's text and the hand-labelled expected edges. The eval runs the worker's `process` (cache hits are free), reduces both sides to sets of (source, relation, target) triples, and counts true positives (in both), false positives (predicted only) and false negatives (expected only). It prints precision and recall per document, per relation, and overall, plus the first few misses so a reviewer can see what kind they are.

Part 6. Configuration, tests, findings, and how to work with the project

6.1 config/schema.sql

```
CREATE TABLE IF NOT EXISTS entities (
  entity_id      TEXT PRIMARY KEY,
  type           TEXT NOT NULL,
  canonical_name TEXT NOT NULL,
  attrs          JSON
);
CREATE TABLE IF NOT EXISTS documents (
  doc_id        TEXT PRIMARY KEY,
  source        TEXT NOT NULL,
  url           TEXT,
  published_at  DATE,
  text          TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS edges (
  edge_id       TEXT PRIMARY KEY,
  src           TEXT NOT NULL,
  rel           TEXT NOT NULL,
  dst           TEXT NOT NULL,
  valid_from    DATE,
  valid_to      DATE,
  props         JSON,
  doc_id        TEXT,
  span          TEXT,
  extractor     TEXT NOT NULL,
  confidence    DOUBLE NOT NULL,
  ingested_at   TIMESTAMP DEFAULT current_timestamp
);
```

The three tables from Part 1.4. Note what is `NOT NULL`: every entity has a type and a name; every edge has endpoints, a relation, an extractor and a confidence. `doc_id` and `span` are nullable because deterministic edges have no document; the invariant that model edges always carry both is enforced by the code that builds them, not by the schema.

6.2 config/sources.yaml

One entry per external input. Each has a `type` (how to download and save it: `xlsx`, `pdf`, `shapefile_zip`, `arcgis_geojson`, `json`, `html`), one `url` or a list of `urls`, and a `normalizer` (which module in `policygraph/normalize/` reads it). Document sources add `doc_source`, the label written into the documents table. The comments record what was observed about each source: the FAIR Plan PDF is fiscal-year policies in force; the ZCTA file is the generalised cartographic version because the full one is 528 MB; the CAL FIRE layers are the state and local responsibility areas with their effective dates; which bulletins cover Contra Costa.

6.3 `config/source_hashes.yaml`, `config/extraction_manifest.yaml`

`source_hashes.yaml` is a list of `{url, sha}` pairs written by `make fetch`; it is committed so a change upstream appears as a diff in a pull request. `extraction_manifest.yaml` is the run the graph is pinned to: run id, schema version, status, and the list of artifact ids to load, plus lists of failed and errored jobs (both empty). Changing the model or prompt produces a new manifest, reviewed like any code change; rolling back is pinning the previous one.

6.4 `config/entities.yaml`

The registry of entities that can only be resolved by name: insurers (keyed by NAIC code), officials, regulations and rate filings. Each has an `id`, a `type`, a `name`, a `description` (what the embedding and the judge see), and `aliases` (exact matches for resolver step 2). ZIPs, bills, fires and moratoria are not listed because rules mint them. Adding a new insurer is one YAML entry.

6.5 `config/resolutions.yaml`

Every answer the service ever gave for a mention that the rules and aliases could not resolve, including "nobody" answers with the nearest candidate and its cosine. Committed, so `make resolve` replays without the service, and so a changed answer shows as a diff. In the current corpus it holds one embedding match, four judge matches and one audit record for a mention the alias table now handles.

6.6 `config/places.yaml`

ZIP → place name for Contra Costa, used only for labels on the map, the timeline and the explorer. County membership comes from CDI's data, never from this file.

6.7 The tests (`tests/`)

Tests run with `make test` in about two seconds and need no network, no key and no data files beyond the config. What each file proves:

- `test_fetch.py`: environment variables are expanded and an unset one fails; conditional headers are built only from validators we have; a 304 leaves the prior record untouched; a source without validators is `cached` unless forced.
- `test_normalize.py`: the FAIR Plan parser reads rows and missing values, rejects a malformed row, and follows the footer when a sixth fiscal year appears; the CDI parser handles both releases and fails on a missing column; the ACS sentinel becomes null; the hazard overlay computes shares and ignores non-wildland; the HTML converter and the date finder work on the three date formats.
- `test_chunk.py`: a short text is one chunk; long text is covered end to end with overlaps.

- **test_validators.py**: a valid edge is kept; a paraphrased span, a ZIP not in the text, a wrong type pair, and reversed dates are each rejected with the right message; a word broken across lines still matches.
- **test_expand.py**: ZIP blocks get the right names, dates, ZIPs and verbatim spans from a synthetic bulletin (including the 9 4572 case, which is correctly *not* read as a ZIP); expansion adds the missing fires and ZIPs, keeps the model's own edges, folds yearless mentions, and marks what it added; the prompt view lists blocks without their ZIPs; the mention-folding rules map every known variant.
- **test_worker.py**: processing produces an artifact with the expected id; a second call is a cache hit and `force` recomputes; a bad span triggers exactly one retry and then a `failed_validation` status; merge keeps the more confident duplicate.
- **test_adapters.py**: the OpenAI-compatible adapter sends the right request and reassembles a streamed response; the embedder returns vectors in input order; transport errors are retried and then succeed.
- **test_resolve_service.py**: the decision rule: clear cosine winner accepted; small margin goes to the judge; judge rejection is unresolved but records the nearest; below the band the judge is never called; candidates are restricted to the mention's type and answers are cached.
- **test_resolve.py**: canonical rules for ZIPs, bills, fires and moratoria; aliases are case-insensitive and typed; pins are used before the service; offline residuals become pending; the original mention string is preserved.
- **test_resolution_set.py**: runs the 53-mention regression set in `tests/resolution_set.yaml` through the offline resolver and fails if any expected id or method differs.
- **test_graph.py**: moratorium windows are filled from the id; explicit dates are kept; readable names for rule-minted ids.
- **test_insights.py**: on a six-ZIP fixture graph: residuals sum to zero per fit and both specs run; the deferral module finds the protected ZIP, the right year, and both controls with the expected sums; the moratorium year is the window midpoint; the FAIR share is computed; the trend includes the FAIR series and the event study reads the right before/during values.

6.8 The extraction quality evaluation (`make eval`)

Four hand-labelled bulletins with 1,232 expected edges. The current numbers: precision 0.992, recall 0.997. The ten false positives are the Lake County Eagle Fire's ZIP list, which the 2019 bulletin prints and then withdraws in a footnote, the parser cannot read footnotes, and the choice was to keep the deterministic rule simple and report the miss. The misses are one ZIP the PDF prints as 9 4572 and the "(Lara, ...)" author credit for SB 824 that the model emits in one bulletin of four.

6.9 What the findings say (`docs/FINDINGS.md`)

The findings document is regenerated from the insight tables and every number in it names the query that produced it. In brief:

1. **Hazard explains little; price explains less.** Very High hazard share is significant but the fit explains 13 % of the variation between ZIPs. Adding home value and income makes the Lafayette and Orinda residuals *larger*: expensive ZIPs are dropped less, statewide, not more.
2. **The FAIR Plan absorbed the county's hazard ZIPs after the public series ends.** The county's FAIR Plan book grew 11.9× from FY2021 to FY2025 against 2.6× statewide, concentrated in Orinda, Lafayette, Blackhawk and Alamo.
3. **Two moratoria covered Contra Costa, and the first deferred rather than prevented.** The 2019 Sky Fire moratorium covered eleven county ZIPs with data. Their insurer-initiated non-renewals went 1,219 → 910 → 2,253 across 2019–2021 (0.75× then 1.85× the 2019 level); the matched control went 0.83× then 1.09×. Lafayette went 209 → 136 → 613, Orinda 213 → 132 → 578.
4. **Timing.** The county's non-renewal rate moves after the 2020 fire declarations (the Sky moratorium expiring); its FAIR Plan book more than triples in the fiscal year the Sustainable Insurance Strategy regulations were finalised, before the January 2025 fires. Annual data, so alignment, not effect sizes.
5. **Actors.** The Commissioner sits on 90 % of shortest paths in the county neighbourhood; no insurer reaches any ZIP because CDI's data names no carrier.

And the limits: no per-insurer ZIP data exists; the insurer/insured split ends in 2021; the two CDI releases disagree by 7–8 % on their overlap; rate decisions post-date every non-renewal count.

6.10 How to run it

```
uv sync --extra dev          # install everything
cp .env.example .env         # fill CENSUS_API_KEY and the model key
make service PORT=8001 &     # extraction API
make worker &                # extraction worker (the only model caller)
make all                      # fetch → normalize → extract → resolve → graph → insights
make app                      # http://localhost:8501
```

Without any key: `make graph`, `make insights` and `make app` work from the committed artifacts and pins. `make test` and `make lint` should be green before any commit; CI runs both.

6.11 How to change it

- **Add a source.** Add an entry to `sources.yaml`; if its shape is new, write a normalizer module with `run(spec, files)` and register it in `normalize/__init__.py`; add a `deterministic_edges` family in `graph.py` if it yields edges; test the parser on a small sample.
- **Add a document.** Add its URL to `cdi_bulletins` or `cdi_press`, run `make fetch normalize extract resolve graph insights`, review the new artifact and any new pins, commit them.
- **Add a relation.** Extend `Rel` and `REL_TYPES` in `models.py`, the enum in `schema/v1.json`, and the prompt; bump the schema version; add a golden example.
- **Add an entity that resolves by name.** One entry in `entities.yaml`.
- **Change the prompt or model.** Edit `prompts/v2.md` or the environment variable, run `make extract` (new prompt hash → new artifact ids), run `make eval`, review the manifest diff.

- **Add a finding.** A module in `insights/` with `compute(con)`, listed in `MODULES`; a tab in `app.py`; numbers into `FINDINGS.md`.

6.12 What is deliberately not there

- Cal-Access campaign contributions (`DONATED_T0`) are in the vocabulary but not ingested; the Secretary of State's export is a multi-gigabyte dump with no per-committee endpoint.
- A human review queue for `pending:` entities; today there are none, and the pins file is the review surface.
- Quarterly resolution for the event study; CDI publishes annual counts.
- Bit-exact reproducibility of model output; hosted models are not deterministic even at temperature 0. What is guaranteed is *replayability*: the committed artifacts, pinned by the manifest, rebuild the same graph forever without a key.